Basic Linear Algebra (융합기초선형대수학)

KNU Math 902

Classnotes

Mark Siggers

These notes are supplemental to a class taught from Gilbert Strang's "Linear Algebra and Learning from Data", it is expected that students have this text. It is herein referred to as 'the text'.

v. May 29, 2026

The text starts with an overview of the theory involved in applications of linear algebra to the neural networks used for large language models and deep learning. It uses the example of 'handwriting recognition' as an illustration. This is a useful overview, and you are encouraged to read it, but be warned that it assumes a certain familiarity with the ideas involved.

The text covers not only the required linear algebra, but also the required probability and statistics, and then brings these together to look at the contruction of neural nets. We will not get this far in this course; our focus is the linear algebra.

The first part of the text is entitled 'Highlights of Linear Algebra'. The point of view of the text is that you have had a class in Linear Algebra. This part gives a non-traditional overview of a first class of linear algebra, emphasising the ideas that will be useful for the goal of neural networks. This is extremely nice if you recall your linear algebra, but again, a little hard to follow if you do not– concepts are often brought up before they are defined, or are not defined at all.

My notes will re-arrange some of the material to try to give a more linear presentation, and will fill in some of the notions that the text assumes that you know. Strang's book 'Linear Algebra and is applications' is an excellent reference for bits that are not covered in this book. .

I Highlights of Linear Algebra

Vectors

A *vector* is a one dimensional array of (usually) real numbers:

$$u = (u_1, u_2, \ldots, u_n) \in \mathbb{R}^n$$

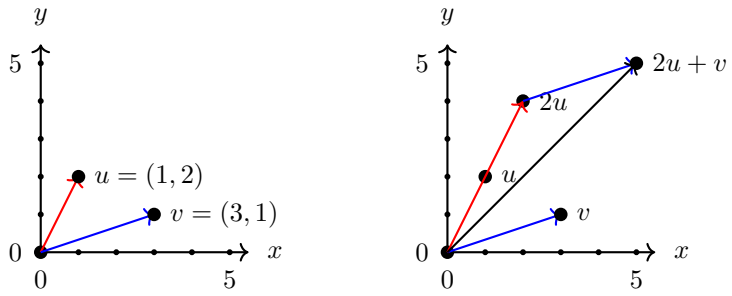
for some $n \geq 1$. We add vectors componentwise

$$(1, 4, 3) + (2, \frac{1}{2}, 0) = (3, 4\frac{1}{2}, 3)$$

and multiply them by scalars in $\mathbb{R}$

$$3(1, 4, 3) = (3, 12, 9).$$

We view vectors either as points in $\mathbb{R}^n$ or as the arrows from the origin to those points:



The *dot product* of vectors $u = (u_1, u_2, \ldots, u_n)$ and $v = (v_1, v_2, \ldots, v_n)$ is

$$u \cdot v = \sum_{i=1}^{n} u_i v_i = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

It is not a vector, rather it is a real number, but it has many nice properties.

You will notice that the dot product $(1, 0) \cdot (0, 1) = 0$ can be 0 for non-zero vectors. The dot product being 0 is actually an interesting case.

Pythagoras tells us that the *length* of a vector $v = (v_1, \ldots, v_n)$ is

$$|v| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \sqrt{v \cdot v}$$

so $v \cdot v = |v|^2$.

We can show more generally, using the cosine rule, that

$$u \cdot v = \cos\theta |u||v|$$

where θ is the angle between u and v. So we get that $u \cdot v = 0$ for non-zero u and v if and only if the angle between u and v is $90°$.

Two vectors u and v are *orthogonal* or *perpendicular*, written $u \perp v$, if $u^T v = 0$.

Vector spaces

A set $X \subset \mathbb{R}^n$ of vectors is a *subspace* of $\mathbb{R}^n$ if it is closed under our operations. That is to say, if for any vectors u and v in X, and any r and s in R, the vector $r \cdot u + s \cdot v$ is also in X.

For a set $X = \{x_1, \dots x_m\}$ of vectors in $\mathbb{R}^n$, the vectors

$$c_1 x_1 + c_2 x_2 + \cdots + c_m x_m$$

for scalars $c_i \in \mathbb{R}$ is a *linear combination* of the vectors in X. The set $\langle X \rangle$ of *linear combinations* of vectors of X is the *subspace generated by* X.

If X is empty, $\langle X \rangle$ contains only the zero vector. As soon as X contains a non-zero vector, then $\langle X \rangle$ contains a line. If we add another vector in that line to X, then $\langle X \rangle$ is unchanged. But if we add any vector not in that line, then $\langle X \rangle$ becomes a plane. One can show that any subspace of $\mathbb{R}^n$ is *isomorphic* to $\mathbb{R}^r$ for some $r \in \{0, 1, \dots, n\}$. This r is called the *dimension* of the subspace.

A set of m vectors in $\mathbb{R}^n$ **can** generate a space of dimension m, but it might generate a smaller space. A set of vectors is *independent* if none of them can be written as a linear combination of the others. The following is a useful alternate definition of independence.

Fact I.1. *A set $\{v_1, \dots, v_m\}$ of vectors is independent if and only if the following implication holds for all choices of real numbers $c_1, \dots, c_m$:*

$$c_1 v_1 + c_2 v_2 + \cdots + c_m v_m = 0 \qquad \Longrightarrow \qquad c_1 = c_2 = \cdots = c_m = 0.$$

A set of r independent vectors in an r-dimensional space generates an r-diminsional subspace of the space, so generates the space. This set of vectors is called a *basis* of the space. A space generally has many different bases, but for the space $\mathbb{R}^n$ the following is usually the nicest.

Example I.2. The set $\{e_1, \dots, e_n\}$ of vectors, where $e_i = (0, 0, \dots, 0, 1, 0 \dots, 0)$ has a 1 in the i^{th} coordinate, is a basis of $\mathbb{R}^n$. It is called the *standard normal basis* of $\mathbb{R}^n$.

A set of vectors is *orthogonal* if any two vectors in the set are orthogonal. The standard normal basis of a space is a orthogonal basis. This is one property that makes it nice. Another is that it is *normal*– each vector in the basis has unit length.

Two subspaces U and V of $\mathbb{R}^n$ are *orthogonal*, written $U \perp V$ if $u \perp v$ for any $u \in U$ and $v \in V$. The orthogonal complement of a subspace U of $\mathbb{R}^n$ is the largest subspace $U^\perp$ of $\mathbb{R}^n$ that is orthogonal to U. Clearly we have that

$$U^\perp = \{x \mid u \cdot x = 0 \text{ for all } x \in U\}.$$

I.1 Multiplication Ax using columns of A

Matrices

An $m \times n$ *matrix* $A = A_{m \times n}$ is a 'rectangle' of numbers consisting of m rows and n columns.

$$A = \begin{bmatrix} 1 & 2 & \frac{1}{2} \\ 4 & 0 & \pi \end{bmatrix}$$

is a 2×3 matrix. We write $A = [a_{i,j}]$ to designate that $a_{i,j}$ is the entry in the i^{th} row (from the top) and the j^{th} column. We drop the comma and write a_{ij} when it does not cause confusion. The values m and n are the *dimensions* of the matrix. A $1 \times n$ matrix is a *(column) vector* and a $m \times 1$ matrix is a *(row) vector*. When $A = [a_{ij}]$ we write a_{i*} for the i^{th} row vector and a_{*j} for the j^{th} column vector. The '$*$' runs over all values.

Practice

Where $A = [a_{ij}]$ is the 2×3 matrix $A = \left[\begin{smallmatrix} 1 & 2 & \frac{1}{2} \\ 4 & 0 & \pi \end{smallmatrix}\right]$, what is a_{23}? What is a_{*2}?

Matrix operations

As with vectors, we add matrices of the same dimensions componentwise

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 3 & 4 \end{bmatrix}$$

and multiply matrices by *scalars* $r \in \mathbb{R}$ componentwise

$$2 \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 4 & 6 \end{bmatrix}.$$

We can also multiply matrices, but only if their dimensions are right. Where $A = A_{\ell \times m} = [a_{ij}]$ and $B = B_{m \times n} = [b_{jk}]$ the product $C_{\ell \times n} = AB$ is the matrix $C = [c_{ik}]$ where

$$c_{ik} = \sum_{j=1}^{m} a_{ij} b_{jk} = a_{i1} b_{1k} + a_{i2} b_{2k} + \cdots + a_{im} b_{mk} = a_{i*} \cdot b_{*k}.$$

The entry $c_{ik} = a_{i*} \cdot b_{*j}$ is the dot product of the i^{th} row of A and the k^{th} column of B:

$$\begin{bmatrix} \cdot & \cdot & \cdot \\ \boxed{1 \quad 4 \quad 2} \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot & \boxed{3} & \cdot & \cdot \\ \cdot & 1 & \cdot & \cdot \\ \cdot & \boxed{1/2} & \cdot & \cdot \end{bmatrix} = \begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & 8 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}.$$

Practice

Compute the two products

$$\begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} x \\ y \\ z \end{bmatrix} \begin{bmatrix} a & b & c \end{bmatrix}$$

and observe that **products do not commute**– the order of the matrices is important when we are multiplying them. These are two different products of two vectors. The first is called the *inner product* or the *dot product*, and the second is called the *outer product*.

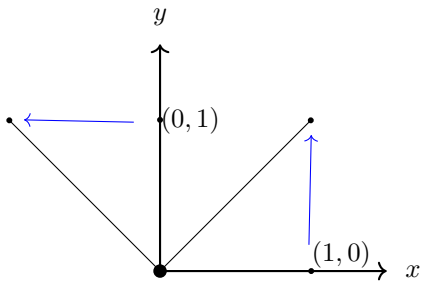
Matrices as linear transformations

Every $m \times n$ matrix A defines a *linear transformation* $T : \mathbb{R}^n \to \mathbb{R}^m$, taking vectors in $\mathbb{R}^n$ to $\mathbb{R}^m$.

$$\begin{bmatrix} 1 & 1 & 2 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 9 \end{bmatrix}.$$

In particular $n \times n$ matrices transform the space $\mathbb{R}^n$. One can usually interpolate the transformation by looking at what it does to a couple of well chosen vectors.

$$\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$



Practice

What is the above transformation doing? Linear transformations T commute with operations: $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$. So they can do such things as stretching (scaling), rotating, reflecting, and projecting.

The identity

The *identity matrix* $I = I_n$ is the $n \times n$ matrix with 1s on the diagonal and 0s everywhere else.

Practice

What transformation does the matrix I_n matrix define? Show that $IA = A = AI$ for any $n \times n$ matrix A.

One of the themes of a basic linear algebra class is deciding when a matrix A has an inverse– a matrix A^{-1} such that $AA^{-1} = I = A^{-1}A$. By dimension considerations, A^{-1} can only exist when A is square; that is, when A is an $n \times n$ matrix. But do all $n \times n$ matrices have inverses? We will look at how to decide this in a second.

(Non square matrices can have left or right inverses, but they will not be the same.)

The transpose

The *transpose* of a matrix $A = [a_{ij}]$ is the matrix $A^T = [a_{ji}]$ that we get from it by transposing rows and columns. A matrix is *symmetric* if $A^T = A$.

Practice

Show that $(A^T)^T = A$ and that $(AB)^T = B^T A^T$. Show that $(A^{-1})^T = (A^T)^{-1}$. Show that for any matrix A the matrix $A^T A$ is symmetric.

As a particular case of this the transpose v^T of a column vector is a row vector, and vice-versa:

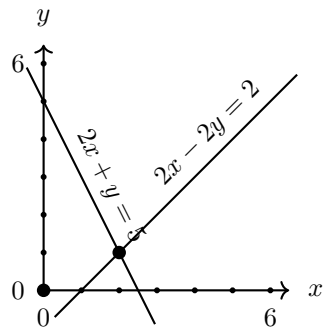
$$\begin{bmatrix} a \\ b \\ c \end{bmatrix}^T = \begin{bmatrix} a & b & c \end{bmatrix}.$$

Thus the inner product $u \cdot v$ is often denoted $u^T v$, as by default we view vectors as column vectors, and the outer product is often denoted uv^T.

The row view of multiplication (From Section I.4 and I.3 of text)

The row view of multiplication arises from the original application of matrices– solving systems of linear equations. Consider the system below, each row of which is an equation that defines a line.

$$\begin{array}{rcrcl} 2x & - & 2y & = & 2 \\ 2x & + & y & = & 5. \end{array}$$



This can be written as a matrix equation

$$\begin{bmatrix} 2 & -2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}.$$

Note: The Row View

More generally, in the *row view of the matrix product* AB, 'the rows of A are coefficients in linear combinations of the rows of B':

$$\begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 3 & 7 \\ 2 & 8 \end{bmatrix} = \begin{bmatrix} 1 \cdot \begin{bmatrix} 3 & 7 \end{bmatrix} + 4 \cdot \begin{bmatrix} 2 & 8 \end{bmatrix} \\ 2 \cdot \begin{bmatrix} 3 & 7 \end{bmatrix} + 3 \cdot \begin{bmatrix} 2 & 8 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 11 & 39 \\ 12 & 38 \end{bmatrix}$$

Remember from highschool how you solved a system of equations like this. Your first step was to 'take a copy of the first equation from the second,' to get an equivalent system of equations.

$$\begin{array}{rcrcl} 2x & - & 2y & = & 2 \\ 2x & + & y & = & 5 \end{array} \quad \rightsquigarrow \quad \begin{array}{rcrcl} 2x & - & 2y & = & 2 \\ & & 3y & = & 3. \end{array}$$

We can do this same reduction with matrices. Multiplying on the left by the matrix $\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$ leaves the first row untouched and removes one copy of the first row from the second:

$$\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

This evaluates to the following matrix equation which represents the reduced system of equations above

$$\begin{bmatrix} 2 & -2 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$

You should understand how to solve matrix equations in this way, using 'row operations' to reduce this to something like

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

from which you can easily read of the solution $(x, y) = (2, 1)$.

This is called Gaussian elimination. You can use *augmented matrices* to reduce your work:

$$\left[\begin{array}{cc|c} 2 & -2 & 2 \\ 2 & 1 & 5 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|c} 2 & -2 & 2 \\ 0 & 3 & 3 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|c} 1 & -1 & 1 \\ 0 & 1 & 1 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|c} 1 & 0 & 2 \\ 0 & 1 & 1 \end{array}\right].$$

In reducing $A = \begin{bmatrix} 2 & -2 \\ 2 & 1 \end{bmatrix}$ to $R = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ we have multiplied on the left by a bunch of *elementary matrices*– matrices that we get from the matrix I by changing one of the 1s to another non-zero number, or changing one of the 0s to a non-zero number. These respectively scale one row of A, or add to a row a scale of another row.

Multiplying all of these together gives a matrix C^* such that

$$C^*A = R.$$

Because R is the identity matrix I, this C^* is actually the *inverse matrix* A^{-1} of A. We can use Gaussian elimination to determine if a matrix A has an inverse, and we can use Gauss-Jordan elimination to actually find A:

$$\left[\begin{array}{cc|cc} 2 & -2 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|cc} 2 & -2 & 1 & 0 \\ 0 & 3 & -1 & 1 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|cc} 1 & -1 & \frac{1}{2} & 0 \\ 0 & 1 & -\frac{1}{3} & \frac{1}{3} \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{6} & \frac{1}{3} \\ 0 & 1 & -\frac{1}{3} & \frac{1}{3} \end{array}\right].$$

In this case we were lucky; our rows, being lines, had exactly one point of intersection, so there was exactly one solution (x, y). This corresponds to A having an inverse. But our lines, (or hyperplanes, when there are more variables) might have empty intersection, or intersect in more than a single point. In this case we could not reduce A to the identity. Our goal would have been reducing A to what is called its *row-reduced echelon form* $R = \text{rref}(A)$ of A:

- The first non-zero entry in any row (the *pivot* of the row) is 1 and is to the right of any pivots above it.
- Any rows of 0 are on the bottom.

Fact I.3. *A matrix A is invertible if and only if* $\text{rref}(A)$ *is the identity. We can find this by Gauss-Jordan elimination.*

Practice

Find the matrix A, the variable vector x, and the real vector b, such that $Ax = b$ is the following system of linear equations.

$$\begin{array}{rcrcrcl} x_1 & + & 2x_2 & - & 3x_3 & = & 3 \\ x_1 & - & 2x_2 & + & 5x_3 & = & 7 \\ x_1 & + & x_2 & + & 7x_3 & = & 9 \end{array} .$$

Use Gaussian Elimination on the augmented matrix $[A \mid b]$ to solve this system.
Use Gauss-Jordan Elimination on the augmented matrix $[A \mid I]$ to find the inverse of A.

The rows of an $m \times n$ matrix A are vectors in $\mathbb{R}^n$, so generate a subspace $R(A)$ of $\mathbb{R}^n$ called the *row space* of A. Gaussian elimination replaces rows with linear combinations of rows, so this space is unchanged.

Fact I.4. *If R is the row-reduced echelon form of A, then A and R have the same row space.*

This is nice, because in R, it is easy to see that the pivot rows are a basis of the row space. The number of non-zero rows in R is the *rank* $r = r(A)$ of A. It will be an important number.

There is another interesting vector space associated to A through the matrix equation $Ax = 0$. The *nullspace* of A is the space

$$N(A) = \{x \in \mathbb{R}^n \mid Ax = 0\}$$

of solutions to the equation $Ax = 0$.

Practice

Show that $N(A)$ is indeed a vector space.

We saw that the set $N(A)$ of solutions of $Ax = 0$ was the intersection of the lines (or hyperplanes– $\mathbb{R}^{n-1}$ subspaces of $\mathbb{R}^n$) corresponding to the rows of A. If A has no non-zero rows, then everything in $\mathbb{R}^n$ is in this intersection of the no rows– the row space has dimension 0 and the nullspace has dimension n. If A has one (non-zero) row, the row space has dimension 1 and the nullspace is the intersection of one hyperplane, so has dimension $n - 1$. It looks like that when the rank of A is r, the row space has dimension r and the nullspace has dimension $n - r$. This is true. And look. For any $x \in N(A)$, we have that $Ax = 0$, showing us that $b \cdot x = 0$ for any row b of A, and so for any vector b in the row space of A. So $N(A)$ is the orthogonal complement of the row space of A.

Fact I.5. *For any $m \times n$ matrix A, where r is the dimension of the row space, $R(A)$, of A, the nullspace, $N(A)$, has dimension $n - r$. Moreover $N(A)^\perp = R(A)$.*

Now take any r-dimensional subspace U of $\mathbb{R}^n$ and make a matrix A whose rows are a basis of U. Then $U^\perp = R(A)^\perp = N(A)$ has dimension $n - r$.

Fact I.6. *For any subspace U of $\mathbb{R}^n$, the dimension of U and its orthogonal complement $U^\perp$ sum to n.*

I.1.1 The column view of Matrix Multiplication

For the column view, we view the columns of a matrix A as vectors in the space $\mathbb{R}^m$. When A is the matrix $A = \left[\begin{smallmatrix} 1 & 3 \\ 2 & 1 \end{smallmatrix}\right]$ its columns are the vectors $u = \left[\begin{smallmatrix} 1 \\ 2 \end{smallmatrix}\right]$ and $v = \left[\begin{smallmatrix} 3 \\ 1 \end{smallmatrix}\right]$. It becomes awkward to always write vectors 'vertically' so we often write vectors as $u = (1, 2)$ and $v = (3, 2)$ or $u = \left[\begin{smallmatrix} 1 & 2 \end{smallmatrix}\right]$ and $v = \left[\begin{smallmatrix} 3 & 2 \end{smallmatrix}\right]$, even when we are thinking of them as columns.

Note: The column view

In the *column view* of multiplication AB we veiw the columns of B as providing coefficients in linear combinations of the columns of A.

$$\begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix}\begin{bmatrix} 3 & 7 \\ 2 & 8 \end{bmatrix} = \begin{bmatrix} 3 & 7 \\ \times & \times \\ \begin{bmatrix} 1 \\ 2 \end{bmatrix} & \begin{bmatrix} 1 \\ 2 \end{bmatrix} \\ + & + \\ 2 & 8 \\ \times & \times \\ \begin{bmatrix} 4 \\ 3 \end{bmatrix} & \begin{bmatrix} 4 \\ 3 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 11 & 39 \\ 12 & 38 \end{bmatrix}$$

In particular, multiplying A by a vector $x = (x_1, x_2)$ yields a vector Ax that is a linear combination of the columns of A:

$$Ax = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 2 \\ 1 \end{bmatrix} = 2 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 1 \cdot \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$$

Often we will ask if a matrix equation $Ax = b$ has a solution. Does

$$\begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

have a solution? Are there values x_1 and x_2 that make this true? The *column space* $C(A)$ of a matrix A subspace of $\mathbb{R}^m$ generated by the columns of A. As a solution to $Ax = b$ is a representation linear combination of columns of A, we have the following.

Fact I.7. *The column space $C(A)$ of A is the set of vectors b for which $Ax = b$ has a solution.*

We avoided defining notation for the row space of A. This is because the book uses the notation $C(A^T)$ for the rowspace of A.

The dimension of the column space of A is related to the dimension of the row space. Indeed we think again about eliminating A to its row-reduced echelon form R. When we do this to solve $Ax = 0$, we know that a solution to $Rx = 0$ is a solution to $Ax = 0$, so columns of A are independent if and only if the corresponding columns of R are independent. The pivot columns are clearly a basis of the column space of R, and so the corresponding columns of A are a basis of its column space. So the column space of A also has dimension r.

Fact I.8. *For a matrix A of rank r, both $C(A)$ and $C(A^T)$ have dimension r. Moveover, we can find a basis for $C(A)$ by eliminating A to its rref R and taking the columns of A corresponding to the pivot columns of R.*

When a matrix A is invertible, so we have an inverse B such that $AB = I_n$, we know that every vector in the standard normal basis is in its column space. And if each of these standard normal vectors is in its column space, then each can be written as a linear combination Ab_i of its columns, so $B = [b_1|b_2|\dots\ b_n]$ is such that $AB = I_n$; and A is invertible. So an $n \times n$ matrix A is invertible if and only if its n columns are independent.

Fact I.9. *A matrix A is invertible if and only if it has rank $r = n$, if and only if it has n independent columns.*

When a matrix A is not invertible, when it doesn't have full rank, (ie. when $r < n$) a basis of the column space is a little harder to find, but we can get one by Gaussian elimination.

Assume that we use row operations to eliminate A to B. Then we know, in particular, that

$$Ax = 0 \qquad \text{and} \qquad Bx = 0$$

have the same solutions. That is, a linear combination of columns of A is 0 if and only if a linear combination of the corresponding columns of B are 0. But B is in row-reduced echelon form, so it is easy to see that the pivot columns are a basis.

Practice

Use Gaussian elimination to find a basis of the column space of $A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$.

I.1.2 The column view of row-reduced echelon form

Finding the echelon form R of a matrix A, we also found the matrix $(C^*)^{-1}$ such that

$$(C^*)^{-1}A = R.$$

This $(C^*)^{-1}$ was a product of row operation matrices, all of which are invertible, so $(C^*)^{-1}$ was invertible; and C^* was its inverse, giving We can then write

$$A = C^*R.$$

Every column of A is a linear combination of columns of C^*, and the pivot columns of R– those columns that are a standard normal vector, correspond to columns of C^* that are exactly columns of A. C^* is invertible, so its columns are independent. The matrix R may have rows of 0 on the bottom; the corresponding columns of C^* are not used the linear combinations expressing the columns of A. Let R^* be the matrix we get from R by removing the rows of zero, and let C be the matrix we get from C^* by removing the corresponding columns. We still get

$$A = CR^*.$$

In the text, this R^* is also called the row-reduced echelon form of A.

Succintly: From Gauss-Jordan elimination we get the decompostion $A = C^*R$. Dropping any rows of zeros from R gives R^*, and dropping corresponding columns from C^* gives C, and we still have the decomposition $A = CR^*$. Both R and R^* are referred to as the row-reduced echelon form of A.

Fact I.10. *Where $A = CR^*$ and R^* is the row-reduced echelon form of A (with rows of zeros removed), the columns of C are a basis of $C(A)$. The rows of R are also independent and so are a basis for the row space of A– the subspace of $\mathbb{R}^m$ generated by its rows.*

In the text, it is suggested how you might find C and R^* without using Gauss-Jordan elimination: For $i = 1$ to m, consider the i^{th} column of A. If it is independent of the set of previous columns, let it be a column of C. Once this is done, each column of A is a linear combination of columns of C, let the columns of R be the coefficients in these linear combinations.

Practice

For the matrix $A = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$, use Gauss-Jordan elimination on the augmented matrix $[A \mid I]$ to find the row-reduced echelon form R of A and the matrix C^* such that $C^*A = R$.
Use the process described above to find C such that $A = CR^*$. Compare C^* and C, and R and R^*.

Problems from the text

I.1: 1, 4, 10, 11, 12 (11 and 12 refer to 10), 13, 16

I.2 Matrix-Matrix Multiplication AB

We have used the inner (dot) product $u^T v$ of vectors a lot, but have said little of the outer product uv^T. This is less emphasised in most linear algebra courses, but it can also be used for defining the matrix product.

Fixing some j, take the j^{th} term $a_{ij}b_{jk}$ of $c_{ik} = a_{i*}^T b_{*k}$. Put this, for all i and k in the i^{th} row and k^{th} column of a matrix C_j– so this is an $\ell \times n$ matrix with ik^{th} entry $a_{ij}b_{jk}$. This matrix C_j is exactly the outer product $a_{*j}b_{j*}^T$.

Thus we can write

$$AB = \sum_{j=1}^{m} a_{*j}b_{j*}^T$$

as a sum of outer product matrices.

The outer product matrices $a_{*j}b_{j*}^T$, have a very simple structure. Each column is a scale of the column a_{*j}, so the matrix has rank 1.

For data science applications, we will often have a matrix A that has some 'important information' and some 'noise'. There will be various techniques for separating these. One of them will be to decompose A as $A = CR$, and then expand CR as a sum of outer product matrices. The 'bigger' outer products will be the important information, and the smaller ones will be the noise.

Some of the problems from the book seem a little open ended. It can be unclear, at first, what they are asking. But with a bit of patience you can get there. And usually, these problems are valuable– the author is insightful, and is leading you to important ideas.

Problems from the text

I.2: 1, 2, 5, 6

I.3 The Four Fundamental Spaces

There are four natural spaces associated to a matrix $A = A_{m\times n}$. We have seen the column space $C(A) \subset \mathbb{R}^m$ and the row space $C(A^T) \subset \mathbb{R}^n$. Both have dimension equal to the rank r of A. We have seen the nullspace $N(A)$, the set of solutions to $Ax = 0$, which is the orthogonal complement of $R(A^T)$, so has dimension $n - r$. The last of the four fundamental spaces is the left nullspace $N(A^T)$ of A. This is the set of x such that $xA = 0$. It is the orthogonal complement of $C(A)$, and has dimension $m - r$. This interpretation of this space is a little less obvious, but lets look at how it, and the other spaces, is sometimes used in applications.

Practice

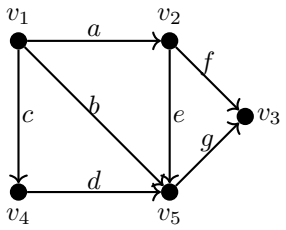
Let B be a matrix you get from A by Gaussian elimination. Which of the four fundamental subspaces change, and which stay the same.

I.3.1 Application to graphs

The incidence matrix M_G of a graph G

Recall that a graph is a set structure consisting of a finite set V of *vertices* and set E of two elements subsets of V called *edges*. There are several ways to represent a graph with a matrix. We will use the (oriented) incidence matrix. For this, we give the graph an arbitrary orientation, replacing an edge $\{u, v\}$ with either the *arc* (u, v) or the arc (v, u).

Example I.11. We can draw a graph G as follows. (We will keep referring back to this graph in most of our examples.)



and can alternately represent it by its incidence matrix M_G:

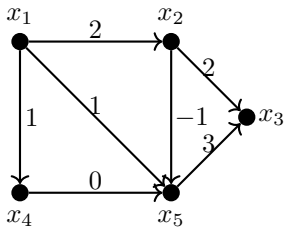
$$
\begin{array}{cc} & \text{Nodes} \\ & \begin{array}{ccccc} v_1 & v_2 & v_3 & v_4 & v_5 \end{array} \\ \text{Arcs} \begin{array}{c} a \\ b \\ c \\ d \\ e \\ f \\ g \end{array} & \begin{bmatrix} -1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 1 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \\ 0 & -1 & 0 & 0 & 1 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 \end{bmatrix} \end{array}
$$

As we usually refer to M_G as an $m \times n$ matrix, m is the number of edges of G, and n is the number of vertices of G. This agrees with the standard naming conventions for graphs! The number of vertices is always n and the number of edges is always m.

Column space of M_G

There is a graph interpretation of the column space of M_G– the set of values b such that $M_G\mathbf{x} = \mathbf{b}$ has a solution.

With our previously pictured G, let $\mathbf{b} = (2, 1, 1, 0, -1, 2, 3)$ and assume that $\mathbf{x}$ is a solution to $M_G\mathbf{x} = \mathbf{b}$. The solution assigns a value x_i to each node v_i, and $\mathbf{b}$ assigns a value to each arc.

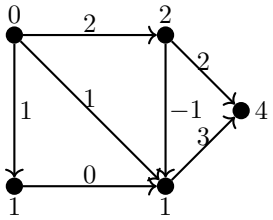


Assume that there is a solution with $x_1 = 0$. (We will see below that if there is a solution, then we can always make this assumption.)

The first row of $M_G\mathbf{x} = \mathbf{b}$, that is, of

$$\begin{bmatrix} -1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 1 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \\ 0 & -1 & 0 & 0 & 1 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 0 \\ -1 \\ 2 \\ 3 \end{bmatrix}$$

says that $-x_1 + x_2 = 2$. Thus the 2 corresponding to the arc $v_1 \to v_2$ means that $x_2 = x_1 + 2$. So a solution looks like this:



It seems we were lucky that this all worked out; but it worked out because the given equation has a solution. When we follow the arcs around a circle, adding values to the vertices according to

the values on the edges, things might not work out. We need that the sum of the arc values around any cycle (paying attention to orientation) is 0.

Practice

Show that if G is a *tree*– it is connected and contains no cycles– then $M_G\mathbf{x} = \mathbf{b}$ has a solution; and indeed has a unique solution for any choice of x_1.

Essentially the same proof gives the following.

Fact I.12. *$M_G\mathbf{x} = \mathbf{b}$ has a solution if and only if the sum of the values in $\mathbf{b}$ corresponding to any cycle c of G, add up to 0.*

This fact appears in all sorts of applications. If your graph is a map, and the values on the arcs are increase in elevation then a solution assigns elevations to the vertices, and clearly if you walk in circuit, from a vertex back to that vertex, your total change in elevation is 0.

It shows up in circuit diagrams as Kirchoff's law too- the values on the vertices are voltages (or potentials), and the values on the arcs are voltage drops (or current, when all arcs have resistance 1). The voltage drop between two nodes is the same whichever path you take– so voltage drop on the circuit make up of two such paths (one traversed backwards) is 0. The text goes a lot deeper into circuit diagrams. I don't understand it though.

The nullity of M_G

The following simple fact is quite useful; it is true because each row of M_G contains a 1 and -1 so the dot product with the vector $\mathbf{1}$ of ones, is $-1 + 1 = 0$.

Fact I.13. *For any graph G, the nullspace $N(M_G)$ of M_G contains the vector $\mathbf{1}$.*

This allows us to say that if an equation $M_G x = b$ has a solution $x = (x_1, \ldots, x_n)$, then it has a parallel one (which we get by adding $(x_1 - a)\vec{1}$) in which $x_1 = a$ for any value a that we find convenient. This justifies our assumption above that there was a solution to $M_G x = b$ with $x_1 = 0$.

Now, what else is in the nullspace of M_G?

Let x be a solution with $x_1 = 1$. If there is an arc from v_1 to v_i, then we get that $x_i = 1$ from the fact that the v_1v_i row of $M_G\vec{1}$ is 0.

In the graph shown above we can then repeat this argument, and show that $x_i = 1$ for all i. So M_G has nullity 1. But this is not true of all graphs.

Practice

Show that for any connected graph G, M_G has nullity 1. Show that in general the nullity of M_G is the number $c(G)$ of components of G:

$$n - r = c(G).$$

Rank Arithmetic

The rank $r(A)$ of a matrix A is an important measure, so we make some easy observations about the rank of derived matrices.

Let A and B be matrices.

- $r(AB) \leq \min(r(A), r(B))$.
- $r(A + B) \leq r(A) + r(B)$.
- $r(A^T A) = r(AA^T) = r(A) = r(A^T)$.
- If $A = A_{m\times r}$, $B = B_{r\times n}$ and both matrices have rank r, then AB has rank r.

Practice

Show that the above are true. (Try to do as many of them as you can, and then look in the text for the solutions.)

Problems from the text

I.3: 1, 2, 3, 5, 6, 9, 11

I.4 Elimination and $A = LU$

To solve $Ax = b$, we reduced a matrix A to $R = \text{rref}(A)$ by *elementary row operations:* scaling rows and adding copies of rows to other rows. Usually we are a bit more methodical about this. We start on the first column, and 'clear' below its first entry. Then we 'clear' below the second pivot, and continue in this way. Clearing below the pivots in this way is called 'elimination'. Once this is done, we continue to clear above the last pivot, and then the previous pivot, etc. This is called 'back substitution'.

To reduce an $n \times n$ matrix A to the identity takes a lot of operations. Every row operation takes n basic operations (additions or multiplication).

Practice

Show that the elimination step of Gaussian elimination takes about n^3 basic operations; and the back substitution step takes another, about, n^2. Show that for Gauss Jordan, both steps take about n^3.

So solving $Ax = b$ via Gaussian Elimination takes about $n^3 + n^2$ operations, the main bottleneck being the elimination. Finding A^{-1} takes about twice as long.

We won't improve this now for $Ax = b$ but we will observe how we can save time if we have to solve this equation for various b. One obvious attack at this is to just go ahead and find A^{-1}. It takes a bit more time, but then we can just solve $Ax = b$ for various b by computing $x = A^{-1}b$. This takes about n^2 basic operations for each b.

But we can do the same thing without computing A^{-1}, by stopping Gaussian elimination after the elimination step.

In one of our previous examples we used Gauss-Jordan elimination to reduce

$$\left[\begin{array}{cc|cc} 2 & -2 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{array}\right] \qquad \text{to} \qquad \left[\begin{array}{cc|cc} 1 & 0 & \frac{1}{6} & \frac{1}{3} \\ 0 & 1 & -\frac{1}{3} & \frac{1}{3} \end{array}\right]$$

If we stop before back substitution, we get

$$\left[\begin{array}{cc|cc} 2 & -2 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|cc} 2 & -2 & 1 & 0 \\ 0 & 3 & -1 & 1 \end{array}\right] =: [U \mid L^{-1}].$$

We have called these matrixes U and L^{-1} because the first is *upper triangular*: all entries below the diagonal, that is, all $u_{ij} = 0$ if $i > j$. The second matrix is lower triangular. Stopping here saves us the n^2 operations of back substitution.

We can use U and L^{-1} to solve $Ax = b$ as follows. We have that $L^{-1}A = U$ and so from $Ax = b$ we get that

$$Ux = L^{-1}Ax = L^{-1}b.$$

Computing $b' = L^{-1}b$ we can solve $Ux = b'$.

This seems like we are back in the orignal position, we must solve $Ux = b'$ instead of $Ax = b$. But solving $Ux = b'$ is quicker, because U is upper diagonal. Solving it is just back substitution–

it takes just n^2 operations. And L^{-1} is lower triangular, so computing $L^{-1}b$ only takes half of the usual n^2 operations.

Practice

Where $A = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 1 & 3 \\ 1 & 2 & 3 \end{bmatrix}$ use Gauss-Jordan elimination to reduce $[A \mid I]$ to $[U \mid L^{-1}]$ and then use this to solve $Ax = b$ where $b^T = [2\ 2\ 3]$.

Practice

The matrix L^{-1} is lower triangular, but it is invertible and L is also lower triangular. Explain why this is true. (Think of how Gauss-Jordan elimination builds L^{-1} buy row matrices.)

When solving for several b, using an LU decomposition of A saves a bit of time– about a factor of 2. That might not seem like much, but if computation takes a week, it is nice. The decomposition into trianglar matrices has other uses which we will see later.

Permutations and $PA = LU$

We took it for granted that we could reduce A to rref(A) by elementary row operations. But can you reduce $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ to rref(A)? It doesn't work, does it.

Sometimes in Gaussian elimination, we have to use a row switch– we switch the positon of two rows. We can effect this by multiplying on the right by a permutation matrix– a matrix with one 1 in each row and column.

If you keep track of the rows you switch, then can switch the rows before starting Gaussian elimination, and it will finish sucessfully. In this case we combine all the row switches into a permutation matrix P. We can then find an LU decomposition of PA.

Problems from the text

I.4: 1, 3, 5, 6, 7, 11 (you will have to read the text to understand 11),

I.5 Orthogonality

We already talked about when vectors or subspaces of $\mathbb{R}^n$ are orthogonal. We talked of the standard normal basis of $\mathbb{R}^n$ being the nicest basis of $\mathbb{R}^n$ partly due to the fact that it is an *orthogonal basis*: any two vectors in the basis are orthogonal. Well any subspace has a basis that is *orthanormal*: both orthogonal and normal. And this has some of the nice features of the standard normal basis.

If $\{q_1, \ldots, q_r\}$ is a basis of a space V, then any vector v in V is, of course, a linear combination of the q_i:

$$v = c_1 q_1 + \cdots + c_r q_r.$$

When the basis is orthanormal, these c_i are very easy to find. Indeed,

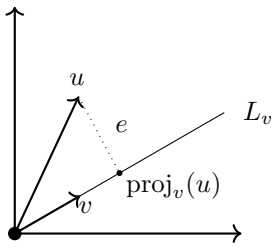
$$q_i^T v = c_1 q_i^T q_1 + \cdots + c_r q_i^T q_r = c_i |q_i|^2 = c_i.$$

We can write any vector v in V as

$$v = (q_1^T v) q_1 + \cdots + (q_r^T v) q_r.$$

Finding an Orthogonal basis by Gram-Schmidt

The projection of a vector u onto a line L is the vector in L that is closest to u. This is the point in the line from which there is a perpendicular bisector that contains u. The *projection* of u onto the vector v is the projection $\text{proj}_v(u)$ of u onto the line L_v generated by v.



One can show that

$$\text{proj}_v(u) = \frac{u^T v}{v^T v} v.$$

Notice that when we subtract $\text{proj}_v(u)$ from u, we are left with the vector $e = u - \text{proj}_v(u)$ from $\text{proj}_v(u)$ to u, called the *error*, which is orthogonal to $\text{proj}_v(u)$. Thus we can write u as the sum

$$u = \text{proj}_v(u) + e$$

of orthogonal vectors.

Given a basis $\{b_1, \ldots, b_m\}$ of a subspace V of $\mathbb{R}^n$ we now find an equivalent orthanormal basis $\{q_1, \ldots, q_m\}$ using the following Gram-Schmidt process. First, let $q_1 = b_1/\|b_1\|$ and for $i = 2, \ldots, m$ let $q_i = q_i'/\|q_i'\|$ where

$$q_i' = b_i - \sum_{j=1}^{i-1} \text{proj}_{q_j} b_i.$$

This q_i' is what is left when we remove from b_i its projection onto the previous vectors in the basis. So it is orthogonal to them.

Example I.14. Let $V \subseteq \mathbb{R}^3$ be generated by

$$B = \{b_1 = (1,0,1), b_2 = (1,0,0), b_3 = (2,1,0)\}.$$

Clearly the standard normal basis is an equivalent basis, but ignoring this for the moment, we use the Gram-Schmidt process to get another orthanormal basis.

- $q_1 = b_1/\|b_1\| = \frac{1}{\sqrt{2}}(1,0,1)$
- $q_2' = b_2 - (q_1^T b_2)q_1 = (1,0,0) - \frac{1}{\sqrt{2}}\frac{(1,0,1)}{\sqrt{2}} = (1/2, 0, -1/2)$
- $q_2 = \frac{1}{\sqrt{2}}(1,0,-1)$
- $q_3' = b_3 - (q_1^T b_3)q_1 - (q_2^T b_3)q_2 = \cdots = (0,1,0)$
- $q_3 = (0,1,0)$.

Projection Matrices

Just like we projected a vector onto a line, onto the space generated by a vector v, we can project a vector in $\mathbb{R}^n$ onto any subspace of $\mathbb{R}^n$. Indeed, we use a similar formula to project a vector u onto the column space of a matrix A. The matrix $P_A = A(A^TA)^{-1}A^T$ is called the projection matrix for A, for any vector u we have that $P_A u$ is the projection of u onto $C(A)$. This means:

- $P_A u$ is in $C(A)$ for all vectors u.
- $u - P_A u$ is orthogonal to $C(A)$ for all vectors u.
- $P_A u = 0$ if u is orthogonal to $C(A)$.
- $P_A u = u$ if u is in $C(A)$, and so $P_A P_A u = P_A u$ for all u. (P_A is idempotent.)

More generally, a matrix P is a *projection matrix* if for some matrix A, the first two properties hold for P_A. One can show that a matrix is a projection matrix if and only if it is symmetric and idempotent.

We will use projections matrices in least squares approximations: when $Ax = b$ doesn't have a solution, this gives us an $\hat{x}$ that is the 'closest thing there is to a solution'.

Orthogonal matrices

Consider doing Gram-Schmidt on the columns of a matrix A. At each step, we replace a column with a linear combination involving earlier columns. The resulting matrix Q with orthonormal columns can be written as $Q = AR^{-1}$ where R is upper triangular. (This will give us what is called the QR decomposition of A.)

A matrix, like Q with orthonormal columns is are nice. We have $Q^TQ = I_n$ and so the (left) inverse $Q^{-1} = Q^T$ is easy to compute. The matrix QQ^T is symmetric, and idempotent: $(QQ^T)(QQ^T) = QQ^T$, which means that it is a projection matrix.

A square matrix Q with orthonormal columns is called *an orthogonal matrix*. These are the nicest of matrices. As transformations of $\mathbb{R}^n$ they preserve lengths and the angle between vectors, so are rigid transformations: rotations and reflections. Indeed

$$(Qu)^T(Qv) = u^TQ^TQv = u^TIv = u^Tv$$

so $|Qv|^2 = (Qv)^T(Qv) = v^Tv = |v|^2$ and the angle $\frac{(Qu)^T(Qv)}{|Qu||Qv|}$ between Qu and Qv is

$$\frac{(Qu)^T(Qv)}{|Qu||Qv|} = \frac{u^Tv}{|u||v|},$$

the angle between u and v.

Problems from the text

I.5: 1, 2, 3, 5, 6, 8

The determinant

The determinant $\det(A)$ (written also $|A|$ or $\left|\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right|$ for a matrix $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$) of a square matrix A is a single number that tells us a lot about A. We can uniquely define it by insisting that the following three properties hold.

1) $\det(I_n) = 1$.

2) A row exchange of A changes the determinant of A by a factor of -1.

3) The determinant is linear in the first row:

$$\begin{vmatrix} tw_1 & tw_2 & tw_3 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{vmatrix} = t \begin{vmatrix} w_1 & w_1 & w_1 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{vmatrix} \qquad \text{and} \qquad \begin{vmatrix} a_1+b_1 & a_2+b_2 & a_3+b_3 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{vmatrix} = \begin{vmatrix} a_1 & a_2 & a_3 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{vmatrix} + \begin{vmatrix} b_1 & b_2 & b_3 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{vmatrix}$$

Notice that the determinant is **not** linear in the whole matrix, just in the first row.

Practice

Using property 2) show that the determinant is linear in any row of the matrix. Show in particular that if we get A' by multiplying a row of A by r, then $|A'| = r|A|$.

We now look at some further properties that will help us compute the determinant.

4) If two rows of A are equal, then $\det(A) = 0$.

5) The row operation of adding a multiple of one row to another doesn't change the determinat.

6) If A has a row of zeros, then $\det(A) = 0$.

Practice

We now know how the determinant acts under row operations, so we can compute the determinant using Gaussian elimination. Try this with the matrix $A = \left[\begin{smallmatrix} 1 & 2 & 3 \\ 2 & 2 & 2 \\ 0 & 0 & 1 \end{smallmatrix}\right]$.

7) If A is triangular then $\det(A)$ is the product of the diagonal entries.

8) A is invertible if and only if $|A| \neq 1$. We call a matrix A *singular* if $|A| = 0$.

9) The determinant is multiplicative: $|AB| = |A||B|$.

10) $|A| = |A^T|$.

Practice

Prove properties 4) - 10) using properties 1) - 3). Hint: if 7,9 and 10) seems hard, make sure do the previous practice problem first.

Formulae for Determinants

We saw how to calculate the determinant by elimination, keeping track of row swaps:

$$\begin{vmatrix} 1 & 2 & 3 \\ 2 & 4 & 4 \\ 1 & 0 & 2 \end{vmatrix} = \begin{vmatrix} 1 & 2 & 3 \\ 0 & 0 & -2 \\ 0 & -2 & -1 \end{vmatrix} = - \begin{vmatrix} 1 & 2 & 3 \\ 0 & -2 & -1 \\ 0 & 0 & -2 \end{vmatrix} = -(1 \cdot -2 \cdot -2) = -4.$$

For a 2×2 matrix $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ we can use the elimination method to get

$$\left|\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right| = \left|\begin{smallmatrix} a & b \\ 0 & d - b\frac{c}{a} \end{smallmatrix}\right| = ad - cb.$$

This is an easy formula to memorise, and quick to use.

Practice

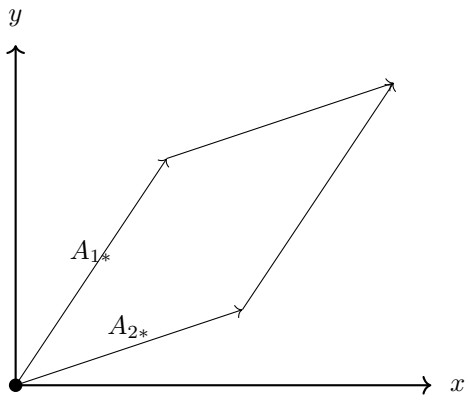
Compute the determinant of $\left[\begin{smallmatrix} 1 & 4 \\ 3 & 1 \end{smallmatrix}\right]$.

Often in a linear algebra class you would look at several different formulas for computing the determinant: the permutation formula, and the co-factor formula. We will skip these for the time being, but recall the useful fact that the inverse of an invertible 2×2 matrix $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ is

$$\left|\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right| = \left[\begin{smallmatrix} d & -b \\ -c & a \end{smallmatrix}\right] / ad - cb.$$

There is a more general version of this for bigger square matrices that one gets from the cofactor formula for the determinant, but it is less useful.

Another cool application of the determinant is that for an $n \times n$ matrix A, the volume of the n-dimensional parallelopided formed by the column vectors of A is exactly $|\det(A)|$.



Practice

A box has edges from $(0,0,0)$ to $(3,1,1), (1,3,1)$ and $(1,1,3)$. Find its volume.

I.6 Eigenvalues and Eigenvectors

An *eigenvalue* of a matrix A is a constant λ such that

$$Ax = \lambda x$$

for some non-zero vector x. The vector x is the corresponding *eigenvector*.

Example I.15. For the matrix $A = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 3 \end{smallmatrix}\right]$, we have

$$A\left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right] \text{ and } A\left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 0 \\ 3 \end{smallmatrix}\right] = 3\left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right],$$

so 1 is an eigenvalue with eigenvector $(1,0)$ and 3 is another eigenvalue with eigenvector $(0,1)$.

Practice

Does $A = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 3 \end{smallmatrix}\right]$ have any more **eigenvectors**?

Indeed $A\left[\begin{smallmatrix} 7 \\ 0 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 7 \\ 0 \end{smallmatrix}\right]$. All vectors in the space $\langle (1,0) \rangle = \{(r,0) \mid r \in R\}$ are eigenvectors with eigenvalue 1.

Practice

Show that the set of vectors that are eigenvectors for a given eigenvalue λ are a space.

This space is the *eigenspace* of λ.

So $A = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 3 \end{smallmatrix}\right]$ has eigenvectors in the spaces $\langle (1,0) \rangle$ and $\langle (0,1) \rangle$.

Practice

Show that A has no other eigenvectors. Indeed show that an $n \times n$ matrix can have at most n eigenvalues.

In this example it was easy to find the eigenvectors by inspection. Sometimes knowing what the matrix does as a transformation can also be used to find the eigenvectors and eigenvalues.

Practice

What are the eigenvectors of the rotation $A = \left[\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right]$?

Finding Eigenvalues algebraically

A more general method for finding the eigens (eigenvalues and eigenvectors) of a matrix A is as follows. To find v such that

$$Av = \lambda v$$

for some λ, we can write this as the matrix equation $Av = \lambda Iv$, and re-arrange this to get

$$(A - \lambda I)v = 0.$$

If $A - \lambda I$ is invertible (= non-singular), then this only has the one solution $v = 0$, which is not useful, so we want λ that make the *characteristic matrix* $A - \lambda I$ of A singular. Such a λ is an eigenvalue, and the nullspace of $A - \lambda I$ is the eigenspace of λ. As a matrix is singular if and only if its determinant is 0, we can find the eigenvalue λ for A as follows.

Example I.16. To find the eigens of $A = \left[\begin{smallmatrix} 2 & 3 \\ 1 & 4 \end{smallmatrix}\right]$ we write out its *characteristic equation*:

$$0 = |A - \lambda I| = \begin{vmatrix} 2-\lambda & 3 \\ 1 & 4-\lambda \end{vmatrix}.$$

Using our determinant formula to expand on the right to find the *characteristic polynomial* of A:

$$\begin{aligned} \begin{vmatrix} 2-\lambda & 3 \\ 1 & 4-\lambda \end{vmatrix} &= (2-\lambda)(4-\lambda) - 3 \\ &= \lambda^2 - 6\lambda + 5 \\ &= (\lambda - 1)(\lambda - 5) \end{aligned}$$

solve the quadratic equation $(\lambda - 1)(\lambda - 5) = 0$ to get $\lambda = 1$ or 5.

We usually write the eigenvalues order of decreasing value (or sometimes decreasing absolute value) as $\lambda_1 = 5$ and $\lambda_2 = 1$.

Practice

Find the eigenspaces of λ_1 and λ_2 by finding the nullspaces of $A - \lambda_1 I$ and $A - \lambda_2 I$.

The same process works for any square matrix A:

- Find eigenvalues λ_i by solving the characteristic equation $0 = |A - \lambda I|$.
- Find the eigenspace of λ_i by finding the nullspace of $A - \lambda_i I$.

Note that the characteristic equation of an $n \times n$ matrix A has n roots. These will usually be hard to find algebraically if $n \geq 4$, so we will have to find numerical ways to find them. There can be repeated roots, and there can be complex roots.

For repeated roots, we get repeated eigenvalues, and an eigenvalue λ with multiplicity r, the eigenspace of λ can have any dimension from 1 to r. This r is the *algebraic multiplicity* of the eigenvalue, while the dimension of its eigenspace is its *geometric multiplicity*.

A real eigenvector (with a real eigenvalue) tells us the direction in which the matrix, as a transformation, stretches (or shrinks). What does a complex eigenvectors mean?

Practice

We asked earlier what the eigenvectors of the rotation matrix $R = \left[\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right]$ are? Likely you said that as it rotates all vectors by 90 degrees, and this not a scale, there are none, so no eigenvalues either. Use the characteristic polynomial of R to find its eigenvalues, (they exist!) and their eigenvectors.

Practice

Show that for a triangular matrix T, the eigenvalues are the diagonal entries.

Practice

If A has an eigenvalue λ, what can you say about the eigenvalues of the matrix $A + cI$? What about BAB^{-1} for some invertible matrice B?

Similar Matrices

If $Ax = \lambda x$ and B is invertible, then for the vector Bx we have

$$BAB^{-1}(Bx) = BAx = B\lambda x = \lambda Bx.$$

So A and BAB^{-1} have the same eigenvalue λ. (Its eigenvector under A is x, and its eigenvector under BAB^{-1} is Bx.) Two $n \times n$ matrices are *similar* if they have the same eigenvalues.

If A has a full set of eigenvectors, that is, n independent eigenvectors $\{v_1, \ldots, v_n\}$ (so all eigenvectors $\{\lambda_1, \ldots, \lambda_n\}$ have the same geometric multiplicity as algebraic multiplicity), then we have

$$A[v_1|v_2|\ldots|v_n] = [\lambda_1 v_1|\ldots|\lambda_n v_n] = [v_1|v_2|\ldots|v_n]\begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{bmatrix}.$$

Letting X be the matrix whose i^{th} column is the i^{th} eigenvector v_i and Λ be the diagonal matrix whose i^{th} entry is λ_i, we get the *diagonalisation*

$$A = X\Lambda X^{-1}$$

of A.

The diagonalisation has some nice applications. We can easily compute powers A^n of A:

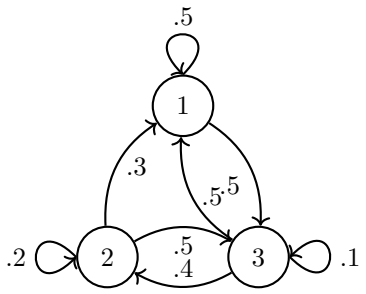
$$A^p = (X\Lambda X^{-1})^p = X\Lambda^p X^{-1} = X\begin{bmatrix} \lambda_1^p & & & \\ & \lambda_2^p & & \\ & & \ddots & \\ & & & \lambda_n^p \end{bmatrix} X^{-1}.$$

We use this in the following example.

Example I.17. The matrix

$$T = \begin{bmatrix} .5 & .3 & .5 \\ 0 & .2 & .4 \\ .5 & .5 & .1 \end{bmatrix}$$

is the transtision matrix for the Markov process described by the following state diagram:



There are three states: $1, 2$ and 3. A *state* vector $u(t) = (p_1(t), p_2(t), p_3(t))$ lists the probability $p_i(t)$ that we are in state i at time t. The state digram, and the transition matrix T, show the probablility t_{ij} that, if in state i at time t, we will be in state j at time $t + 1$.

So

$$u(t + 1) = Tu(t),$$

and for some initial state $u(0)$ we have that

$$u(t) = T^t u(0),$$

To 'solve' the Markov Process, we want to find the limiting state vector $u(t)$ as t goes to infinity.

Well we can find the eigens of T:

$$\begin{aligned} \lambda_1 &= -2/3 & v_1 &= (1, 2, -3) \\ \lambda_2 &= 1/5 & v_2 &= (1, -1, 0) \\ \lambda_3 &= 1 & v_3 &= (13, 5, 10) \end{aligned}$$

We notice that if $p(0) = \frac{1}{28}(13, 5, 10)$ our initial state has eigenvalue one, and so we stay in that state for ever. But what happens if we start in some other state?

To see, we first diagonalise T:

$$T = S\Lambda S^{-1} = \frac{1}{84} \begin{bmatrix} 1 & 1 & 13 \\ 2 & -2 & 5 \\ -3 & 0 & 10 \end{bmatrix} \begin{bmatrix} -2/3 & & \\ & 1/5 & \\ & & 1 \end{bmatrix} \begin{bmatrix} 10 & 10 & -18 \\ 35 & -49 & -21 \\ 3 & 3 & 3 \end{bmatrix}.$$

Then we can compute $u(t) = T^t u(0) = X\Lambda^t X^{-1} u(0)$:

$$
\begin{aligned}
u(t) &= \begin{bmatrix} 1 & 1 & 13 \\ 2 & -2 & 5 \\ -3 & 0 & 10 \end{bmatrix} \begin{bmatrix} -2/3 & & \\ & 1/5 & \\ & & 1 \end{bmatrix} \begin{bmatrix} 5/42 & 5/42 & -3/14 \\ 5/12 & -7/12 & -1/4 \\ 1/28 & 1/28 & 1/28 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} \\
&= \begin{bmatrix} 1 & 1 & 13 \\ 2 & -2 & 5 \\ -3 & 0 & 10 \end{bmatrix} \begin{bmatrix} -2/3 & & \\ & 1/5 & \\ & & 1 \end{bmatrix} \begin{bmatrix} 5p_1/42+5p_2/42-3p_3/14=:c_1 \\ 5p_1/12-7p_2/12-1p_3/4=:c_2 \\ 1p_1/28+1p_2/28+1p_3/28=:c_3 \end{bmatrix} \\
&= c_1(-2/3)^t \begin{bmatrix} 1 \\ 2 \\ -3 \end{bmatrix} + c_2(1/5)^t \begin{bmatrix} 1 \\ -2 \\ 0 \end{bmatrix} + c_3(1)^t \begin{bmatrix} 13 \\ 5 \\ 10 \end{bmatrix}
\end{aligned}
$$

Now as t gets big, the first two disappear, and so

$$
u_t \to c_3 \begin{bmatrix} 13 \\ 5 \\ 10 \end{bmatrix} = \frac{1}{28} \begin{bmatrix} 13 \\ 5 \\ 10 \end{bmatrix},
$$

which is our steady state. We don't even have to compute c_3, as we know that this must be a state vector, so we can scale it to make it one. The initial position $u(0)$ didn't even matter. The other two terms go to 0, and so c_3 is non-zero and we will definitely converge to the state go to the state $\frac{1}{28}(13, 5, 10)$ that we got by normalising the largest eigenvector.

Diagonalizing A used that it had n independent eigenvectors. This can only be done if each eigenspace has the same geometric dimension as algebraic dimension.

Problems from the text

I.6: 1, 2, 3, 4, 5, 6, 7, 8, 11, 20, 22

I.7 Symmetric Positive Definite Matrices

Symmetric matrices S, matrices with $S^T = S$, are nice. We can show that their eigenvalues and eigenvectors are real, and that the eigenvectors can be chosen to be pairwise othogonal. Thus in the diagonalisation $S = X\Lambda X^{-1}$, the matrix X can be taken to be what is called an orthogonal matrix– its columns are normal and pairwise orthagonal. Usually we denote a orthogonal matrix by Q. They have the nice property that $QQ^T = I$, and so $Q^{-1} = Q^T$.

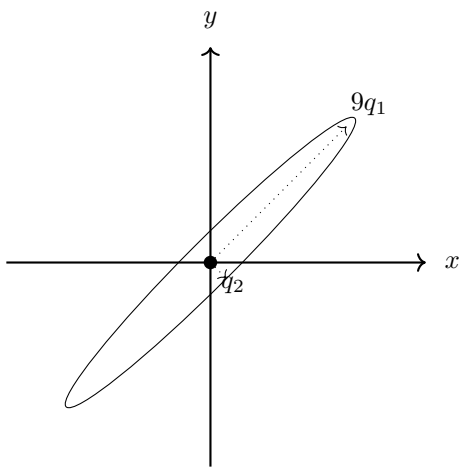
Fact I.18. *Every real symmetric matrix S has a diagonalisation $S = Q\Lambda Q^T$ by an orthogonal matrix Q.*

This gives a nice picture of S as a transformation. Consider applying S to the unit sphere: the set of vectors x with length $x^T x = 1$. What does it do to this sphere? The transformation is linear, so it transforms it to an ellipsoid. We can sketch this ellipsoid by viewing the action on the orthogonal eigenvectors $\{q_1 \ldots, q_n\}$. It stretches q_i by its eigenvalue to $Sq_i = \lambda_i q_i$ so takes the unit sphere to the ellipsoid where the q_i axis has been stretched by λ_i.

Take, for example, $S = \left[\begin{smallmatrix} 5 & 4 \\ 4 & 5 \end{smallmatrix}\right]$ which has eigens

$$\lambda_1 = 9 \qquad q_1 = (\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$$
$$\lambda_2 = 1 \qquad q_2 = (\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}})$$

This transforms the unit 2-dimensional sphere to the ellipse intersecting the q_2 axis at ± 1 and the q_1 axis at ± 9.



Positive Definite Matrices

A matrix is *positive definite* if all of its eigenvalues are positive real numbers, it is *positive semidefinite* if all of its eigenvalues are non-negative reals.

Practice

Show that real $A = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ is positive definite if and only if $a > 0$ and $ac > b^2$.

For a vector v and a matrix A, the value $v^T Av$ is an *energy* of A. When x is an normalised eigenvector with eigenvalue λ then

$$x^T Ax = x^T \lambda x = \lambda |x|^2 = \lambda,$$

so energies can be seen as generalised eigenvalues.

When S is symmetric, so the eigenvectors $x_1, \ldots, x_n$ can be assumed to be orthonormal, any vector v can by written as

$$v = c_1 x_1 + \cdots + c_n x_n$$

and so

$$\begin{aligned} v^T Sv &= (c_1 x_1 + \cdots + c_n x_n)^T S(c_1 x_1 + \cdots + c_n x_n) \\ &= (c_1 x_1 + \cdots + c_n x_n)(c_1 \lambda_1 x_1 + \cdots + c_n \lambda_n x_n) \\ &= c_1^2 \lambda_1 + \cdots + c_n^2 \lambda_n. \end{aligned}$$

This is positive if the eigenvalues are positive.

Fact I.19. *A symmetric matrix is positive definite if and only if all energies are positive.*

This seems a silly way to check if the eigenvalues are positive, but it has its uses. We don't have a nice way to get the eigenvalues of $A + B$ from those of A and B, but the energies of $A + B$ are the sums of the corresponding energies of A and B. So we get the following:

Fact I.20. *If A and B are positive definite symmetric matrices, then so is $A + B$.*

This yields several other characterisations of positive definite matrices. For an $n \times n$ matrix A, the *upper left matrix* A_i of A is the matrix we get from A by removing all but the first i rows and the first i columns. Its determinanat is the i^{th} *upper left determinant* D_i of A. The i^{th} pivot p_i of A is the i^{th} pivot one gets from Gaussian Elimination by basic row operations– no scaling and no row switches.

Using determinant formulas, one can show that $p_i = D_i / D_{i-1}$.

Fact I.21. *For a matrix S the following are equivalent.*

i. S is positive definite.

ii. $S = A^T A$ for some matrix A with independent columns.

iii. All pivots of S are positive.

iv. All upper left determinants D_i of S are positive.

Positive Definite matrices and Minimum problems

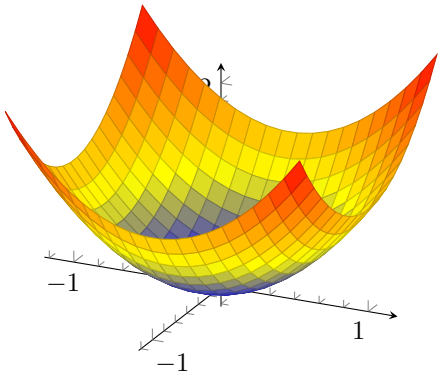
The *energy function* of a matrix S is $x^T Sx$ where x is a vector of variables. When S is symmetric it gives a symmetric polynomial:

$$f_S(x,y) = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = ax^2 + (b+c)xy + dy^2.$$

Problem

Show that for any non-symmetric matrix A there is a symmetric matrix S with the same energy function.

When S is positive definite, the graph of $f_S(x,y)$ is strictly positive– an upwards opening bowl.



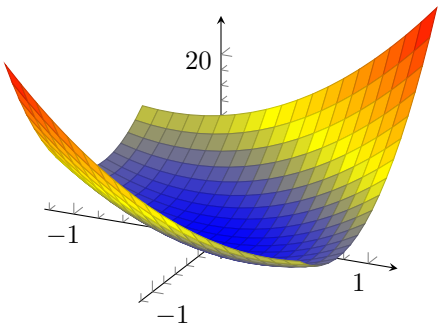
The minimum point of the bowl is where the energy is the smallest. If S has a negative eigenvalue, then the graph is a saddle point, and has negative values. If all eigenvalues are negative, it is a bowl opening down. One can show this with some simple calculus, or by taking slices of the graphs of $z = f_S(x)$. If it is a bowl above $z = 0$ all positive slices that are not empty should be an ellipse.

I.7.1 The ellipse $f_S(x) = 1$

Again using $S = \left[\begin{smallmatrix} 5 & 4 \\ 4 & 5 \end{smallmatrix}\right]$ with eigens

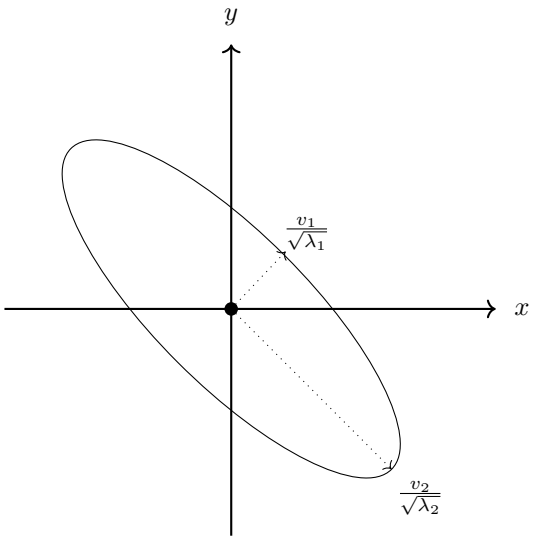
$$\lambda_1 = 9 \qquad q_1 = (\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$$
$$\lambda_2 = 1 \qquad q_2 = (\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}),$$

the energy equation $f_S(x,y) = 5x^2 + 8xy + 5y^2$ is a squeezed paraboloid opening up.



When we intersect with a hyperplane $z = 1$ we get an ellipse

$$5x^2 + 8xy + 5y^2 = 1.$$



As the radius of the elliptical slice is small, the largest eigenvalue point in the direction that $f_S(x)$ is increasing the fastest.

Is it clear why the eigenvalues show up like that? Think of when $S = I$, the ellipse is the unit sphere

$$x_1^2 + x_2^2 + \ldots x_n^2 = 1.$$

When $S = \Lambda$ is a diagonal matrix of eigenvalues λ_i, the ellipse becomes

$$\lambda_1^2 x_1^2 + \ldots \lambda_n^2 x_n^2 = 1.$$

This stretches ellipse by a factor of $\frac{1}{\sqrt{\lambda_i}}$ in the e_i direction.

But really $S = Q\lambda_n Q^T$. This takes the i^{th} eigenvector v_i to e_i first, stretches it, then takes it back to v_i. So it stretches by $\frac{1}{\sqrt{\lambda_i}}$ in the v_i direction.

The Matrix Norm $||A||$

From this we see that the largest eigenvalue tells us the limit of how much a symmetric positive definite A can stretch a vector: $\frac{|Ax|}{|x|} \leq |\lambda_n|$. We will use this notion of how much a matrix can stretch a vector again. It is called the *norm* of the matrix:

$$\|A\| = \max_x \frac{|Ax|}{|x|}.$$

We saw that $\|A\| = \lambda_n$ if A is symmetric positive definite. It turns out that this holds for all square matrices A.

Problems from the text

I.7: 1,2,3,5,14,15,16

I.8 Singular Values and Singular Vectors in the SVD

When A is symmetric we have real eigenvalues and can choose our eigenvectors to be orthogonal, getting our diagonalisation $A = Q\lambda Q^T$ by the orthogonal matrix Q of eigenvectors.

Usually our data isn't symmetric, nor even square, so we don't have eigenvalues and eigenvectors, and we don't have this nice orthogonal decomposition.

In this section we look at singular values and singular vectors– a sort of generalised version of eigenvalue and eigenvectors. We then use them to get a analogue of the diagonalisation of A, called the *singular value decomposition* (SVD) of A.

Singular values and vectors by maximising stretch

The largest eigenvalue of A was the maximum value of $\frac{|Ax|}{|x|}$; it was acheived at the corresponding eigenvalue. It seems reasonable to consider, for non-square A, the vector x that maximizes $\frac{|Ax|}{|x|}$. This vector also maximises

$$\frac{|Ax|^2}{|x|^2} = \frac{(Ax)^T Ax}{x^T x} = \frac{x^T A^T Ax}{x^T x}.$$

This is the normalised energy of the symmetric matrix $S = A^T A$; the maximum value it takes is the first eigenvalue λ_1 of S, and so the maximum value that $\frac{|Ax|}{|x|}$ can take is $\sigma_1 = \sqrt{\lambda_1}$. This is our first *singular value* of A. It occurs at the corresponding eigenvalue v_1 of $S = A^T A$; this is our first *singular vector*.

To find the second singular value, one can then consider maximisiing $\frac{|Ax|}{|x|}$ over all x that are orthogonal to v_1. Again one gets the second singular value $\sigma_2 = \sqrt{\lambda_2}$ at the second singular vector v_2, the second eigenvector of $A^T A$.

Singular value decomposition

We cannot diagonalize $A = Q\Sigma Q^T$ by orthogonal matrixes Q^T consisting of singular values– the dimensions don't work out. The *singular value decomposition* (SVD) of an $m \times n$ matrix A of rank r, is a diagonalisation of A by two different orthogonal matrices U and V

$$A = U\Sigma V^T = \begin{bmatrix} | & & | \\ u_1 \cdots u_m \\ | & & | \end{bmatrix} \left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \cdots & & \\ & & \sigma_r & 0 \\ \hline & 0 & & 0 \end{array}\right] \begin{bmatrix} | & & | \\ v_1 \cdots v_n \\ | & & | \end{bmatrix}^T.$$

Here we necessarily have that U is $m \times m$, Σ is $m \times n$, and V is $n \times n$. To find this decomposition, we look again at the eigens of the symmetric matrix $A^T A$, but also of AA^T.

In fact, if $A = U\Sigma V^T$ then

$$AA^T = U\Sigma V^T V \Sigma^T U^T = U\Sigma\Sigma^T U^T \qquad \text{and} \qquad A^T A = V\Sigma^T \Sigma V^T.$$

So the non-zero entries of Σ are the square roots of the eigenvalues of both U and V– the *singular values* of A. The *singular vectors* corresponding to a singular value σ_i are the i^{th} eigenvector u_i of AA^T and v_i of A^TA.

So, perhaps surprisingly, we have the following.

Fact I.22. *AA^T and A^TA have the same non-zero eigenvalues.*

Rewriting the SVD as

$$AV = U\Sigma \qquad A\begin{bmatrix} | & | \\ v_1 \cdots v_n \\ | & | \end{bmatrix} = \begin{bmatrix} | & | \\ u_1 \cdots u_m \\ | & | \end{bmatrix} \left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \cdots & & \\ & & \sigma_r & 0 \\ \hline & 0 & & 0 \end{array}\right],$$

we see that the first r columns of V dot with rows of A to give non-zero values, so are in its row space, and being orthogonal, are a basis of it. The other $n-r$ columns are in its nullspace. The first r columns of U generate the column space of A.

The columns generating the nullspace of V are not uniquely determined. They are zeroed out in the product by the zeros in Σ. So they, and the 'null' columns of U, are sometimes dropped to give the *reduced SVD* with the $r \times r$ matrix Σ_r:

$$AV_r = U_r\Sigma_r \qquad A\begin{bmatrix} | & | \\ v_1 \cdots v_r \\ | & | \end{bmatrix} = \begin{bmatrix} | & | \\ u_1 \cdots u_r \\ | & | \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \cdots & \\ & & \sigma_r \end{bmatrix}.$$

As an example, the matrix $A = \left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right]$ has singular value decomposition

$$\left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right] = \frac{1}{\sqrt{10}}\left[\begin{smallmatrix} 1 & -3 \\ 3 & 1 \end{smallmatrix}\right]\begin{bmatrix} 3\sqrt{5} & \\ & \sqrt{5} \end{bmatrix}\left[\begin{smallmatrix} 1 & 1 \\ -1 & 1 \end{smallmatrix}\right]\frac{1}{\sqrt{2}}.$$

The outer product decompositon of this is

$$\left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right] = \frac{3\sqrt{5}}{\sqrt{2}\sqrt{10}}\left[\begin{smallmatrix} 1 \\ 3 \end{smallmatrix}\right]\left[\begin{smallmatrix} 1 & 1 \end{smallmatrix}\right] + \frac{\sqrt{5}}{\sqrt{10}\sqrt{2}}\left[\begin{smallmatrix} -3 \\ 1 \end{smallmatrix}\right]\left[\begin{smallmatrix} -1 & 1 \end{smallmatrix}\right] = \frac{3}{2}\left[\begin{smallmatrix} 1 & 1 \\ 3 & 3 \end{smallmatrix}\right] + \frac{1}{2}\left[\begin{smallmatrix} 3 & -3 \\ -1 & 1 \end{smallmatrix}\right].$$

This outer-product decomposition of the SVD decomposition is important in data analysis. It decomposes A as

$$A = \sigma_1 u_1 v_1^T + \cdots + \sigma_r u_r v_r^T.$$

Ordering the singular values so that $|\sigma_1| \geq |\sigma_2| \geq \cdots \geq |\sigma_r|$, the partial sum

$$A_i = \sigma_1 u_1 v_1^T + \cdots + \sigma_i u_i v_i^T$$

is a good rank i approximation to A. We talk about this in the next section.

Problem

Show that if A is symmetric, its SVD is its $Q\Lambda Q^T$ decomposition.

Problem

Show that if A is square, then its largest eigenvalue is at most its largest singular value.

Geometry of the singular values and vectors

Think of transforming the unit ball by $A = U\Sigma V^T$. As V^T is orthogonal, it rotates the unit ball (which doesn't change it at all). Then Σ stretches it into an r-dimensional ellipse. U then rotates this ellipse again. The largest singular value σ_1 tells us how long a vector can stretch. The corresponding singular vectors v_1 and u_1 tell us what stretches like this: v_1 stretches by a factor of σ_1 and then is rotated to point in the direction of u_1.

Fact I.23. *The first singular vector v_1 maximises the value $\frac{|Ax|}{|x|}$, so the norm $\|A\|$ is $|\sigma_1|$.*

Problem

For a matrix $A = U\Sigma V^T$, the *block* matrix

$$S = \begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}$$

is also symmetric. Show that its eigen values are $\pm\sigma_i$ where σ_i are the singular values, and its eigenvectors are $u_i \pm v_i$ where these are the singular vectors.

Problems from the text

I.8: 1, 2, 5, 6, 17

I.9 Principal Components and the Best Low Rank Matrix

We said that matrix

$$A_i = \sigma_1 u_1 v_1^T + \cdots + \sigma_i u_i v_i^T$$

is a good approximation to $A = U\Sigma V^T$.

In fact, the Eckart-Young Theorem says the following.

Theorem I.24. *For a rank r matrix A, and $k \leq r$ the rank k matrix B that minimizes $||A - B||$ is $B = A_k$.*

This is actually true for various norms, and the text gives several proofs for the different norms. We will put off all these proofs, at least for the time being, until we see that they are needed.

The singular vectors u_i and v_i of a matrix $A = U\Sigma V^T$ are also called its *principal components*. Principal Component Analysis (PCA) uses them in a particular 'data' application.

Consider the case where the columns of A_0 are a sample of data vectors. Each row has a particular meaning in the data– the subject's height or income or favourite colour. We can shift them by summing the columns, averaging, and subtracting from A_0 to get a matrix

$$A = A_0 - \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \frac{1}{n}$$

whose rows sum to 0. The average column is then the 0 vector. Plotting the points in $\mathbb{R}^n$ one often gets something close to a line or plane. The first singular vector will best approximate that line. Or the first and second will best approximate that plane.

When columns are sample points of a random sample, the shift from A_0 to A shifts the aspects of the sample to a mean of 0. In this case the diagonal entries of AA^T are the variences and the other entries of AA^T are covariances. A high covariance means that the two aspects of the data are related, so the sample points will tend to cluster in a line in the plane that those aspects make.

Problems from the text

I.9:

I.10 Rayleigh Quotients and Generalized Eigenvalues

The singular values of A were the values we found by maximizing $\frac{x^T S x}{x^T x}$ for the matrix $S = A^T A$. For a symmetric matrix S the *Rayleigh quotient* is the function

$$R(x) = \frac{x^T S x}{x^T x}.$$

Its maximum value is the largest eigenvalue λ_1 of S occuring at $x = q_1$ and it minimum value is the smallest eigenvalue λ_n of S, occuring at $x = q_n$. Again, a bit of calculus shows the following, which tells us that the other eigenvectors give us saddle points.

Theorem I.25. *The critical points of $R(x)$ (where all partial derivatives $\frac{\partial R}{\partial x_i}$ of $R(x)$ are 0) occur at the eigenvectors of S.*

Proof. The energy equation

$$f_S(x) = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix} \begin{bmatrix} & S & \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix} = \sum_{i=1}^{n} \sum_{j=1}^{n} s_{ij} x_i x_j$$

has partial derivative with respect to x_α

$$(f_S)_{x_\alpha} = 2 \sum_{i=1}^{n} s_{\alpha i} x_i = 2 S_{\alpha *} \cdot x = 2(Sx)_\alpha.$$

In particular, the bottom of the Rayleigh quotient is $f_I(x)$ so has partials $(f_I)_{x_\alpha} = 2x_\alpha$.

Using the quotient rule and setting all partials to 0 we get $0 = R_{x_\alpha}$ when we have, for all α, that

$$0 = f_I (f_S)_{x_\alpha} - f_S (f_I)_{x_\alpha} = x^T x \cdot 2(Sx)_\alpha - x^T S x \cdot 2x_\alpha.$$

Rearranging, this holds if and only if

$$\frac{(Sx)_\alpha}{x_\alpha} = \frac{x^T S x}{x^T x} =: \lambda$$

for α which means that $Sx = \lambda x$. □

I.10.1 Generalised Rayleigh Quotient

For many applications we will want to maximize (or minimize) a generalised Rayleigh quotient

$$R(x) = \frac{x^T S x}{x^T M x}$$

where M is another symmetric matrix. Apparently it will often be a 'mass matrix' or 'inertia matrix' in dynamical applications, or a co-variance matrix in statistical applications.

It will be maximised at the first eigenvector of $M^{-1}S$– a vector x that satisfies

$$Sx = \lambda Mx$$

for some constant λ. If M is positive definite, we will not get that the eigenvectors are orthogonal, but we will get that they are M-orthogonal:

$$x_1^T M x_2 = 0$$

In this case, we will be able to get, where $S = A^T A$ and $M = B^T B$ for matrices A and B with the same number of columns, a factorisation

$$A = U_A \Sigma_A Z \qquad \text{and} \qquad B = U_B \Sigma_B Z$$

for the an invertible matrix Z, orthogonal matrices U_A and U_B and positive diagonal matrices Σ_A and Σ_B.

Problems from the text

I.10:

I.11 Norms and Vectors and Functions and Matrices

A norm is a non-negative function on vectors that has many properties similar to what you would expect of a 'length' function. Technically, a *(vector) norm* is a non-negative function $||\ || : \mathbb{R}^n \to \mathbb{R}^+$ that satisfies, for all $r \in \mathbb{R}$ and $v, w \in \mathbb{R}^n$,

- $||r \cdot v|| = |r| \cdot ||v||$, (scaling), and
- $\|v + w\| \leq \|v\| + \|w\|$, (the triangle inequality).

For any $p \geq 1$ the function

$$\|v\|_p = (\sum_{i=1}^{n} |v_i|^p)^{1/p}$$

is a norm. The most common cases we consider are when $p = 1$ or 2, or one defined by taking $p \to \infty$:

- the ℓ^1- or 1-*norm* $\|v\|_1 = |v_1| + |v_2| + \cdots + |v_n|$,
- the ℓ^2- or *Euclidean norm* $\|v\| = \|v\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$,
- the ℓ^∞- or *max norm* $\|v\|_\infty = \max\{|v_1|, |v_2|, \ldots, |v_n|\}$.

Problem:

Show that $\|\ \|_1, \|\ \|_2$ and $\|\ \|_\infty$ are norms.

Problem:

For each of the above norms $\|\ \|$, draw the unit sphere $S = \{v \colon \|v\| = 1\}$ in $\mathbb{R}^2$.

A common problem is to minimise or maximize a norm on the set of vectors in a subspace.

Problem:

Minimise the above norms on the vectors in the line $L : x/3 + y/4 = 1$.

When you do this, you will notice that the minimum point for the ℓ_1-norm is mostly 0s in such problems. While the other norms tend to have non-zero numbers in more coordinates. Because of this the ℓ_1-norm will often be a useful measure in approximations.

I.11.1 S-norms

For any symmetric positive definite matrix the S-norm $\|\ \|_S$ is defined via an S-inner product

$$(v, w)_S = v^T S w$$

by

$$\|v\|_S = (v, v)_S = v^T S v.$$

This is essentially the ℓ_2-norm but with a weighting of the vectors by their 'stretch' under S. This will be used when certain components are considered more important than others.

I.11.2 The Frobenius Norm

We also define norms $\| \ \|$ for matrices. These must satisfy a couple more properties.

i. $\|A\| > 0$ when $A \neq 0$ (positive definiteness)

ii. $\|cA\| = |c|\|A\|$ and $\|A + B\| \leq \|A\| + \|B\|$.

iii. $\|AB\| \leq \|A\|\|B\|$ (sub-multiplicativity)

Viewing an $m \times n$ matrix $A = [a_{ij}]$ as a vector of length mn:

$$(a_{11}, \ldots, a_{1n}, a_{21}, \ldots, a_{2n}, \ldots, a_{m1}, \ldots, a_{mn})$$

we get a matrix norm from a vector norm as long as it satisfies property iii.

Using the ℓ_2 vector norm, one can show that iii. is satisfied, and so one gets the *Frobenius norm*

$$\|A\|_F = \|(a_{11}, \ldots, a_{1n}, a_{21}, \ldots, a_{2n}, \ldots, a_{m1}, \ldots, a_{mn})\|_2.$$

This is easy to compute, and one can show that $\|A\|_F^2$ is the sum of the eigenvalues of $A^T A$, so is the sum of the squares of the singular values of A.

I.11.3 Matrix norms from vector norms

A more universal way to construct a matrix norm from a vector norm gives property iii. immediately. For any vector norm $\| \ \|$ let

$$\|A\| = \max_x \frac{\|Ax\|}{\|x\|}$$

be the maximum stretch of x by A in this norm.

Problem:

Show that property iii. is satisfied above for any choice of the vector norm $\|\|$.

Problem:

Show that $\|A\|_2$ is the largest singular value σ_1 of A, that $\|A\|_1$ is the max of $\|c\|_1$ over all columns c of A, and that $\|A\|_\infty$ is the max of $\|r\|_1$ over all rows of A. Conclude that

$$\|A\|_\infty = \|A^T\|_1 \qquad \text{and} \qquad \|A\|_2^2 \leq \|A\|_1 \|A\|_\infty.$$

I.11.4 The Nuclear norm

One final norm we mention, as it appears useful, is the *nuclear norm*

$$\|A\|_N = \sum_{i=1}^{r} \sigma_i$$

which is defined from the singular values of A. So we have that $\|A^T A\|_N = \|A\|_F^2$.

Problems from the text

I.11:

I.12 Factoring Matrices and Tensors: Positive and Sparse

For matrices A, the SVD $A = U\Sigma V^T$ gives a good understanding of the matrix as a transformation, and with the Eckart-Young Theorem, gives the best rank k approximation of A:

$$A \approx \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_k u_k v_k^T.$$

Absorbing the diagonal Σ into one or both factors, we can simply write our approximation with the factorisation $A \approx UV$.

This allows us to construct columns of A as linear combinations of columns of U. When U has few columns, there are obvious computational advantages, but also obvious limitations– we generally must accept nonzero error $\|A - UV\|$ in constructing the columns of A.

If low rank is all we are interested, or low rank and orthogonal columns in U and V, then Eckart-Young says that SDV will minimize $\|A - UV\|$ for us, for several different norms.

For many applications there are a couple more properties that we would like in the factorisation $A \approx UV$, even at the expense of losing orthogonality.

We will discuss these now, but for most of them, we still want low rank, and will still want to minimize error. So we will look at more general methods to minimize $\|A - UV\|_F$ with various restrictions on U and V.

I.12.1 Nonnegative Matrix Factorisation (NMF)

A matrix $A = [a_{ij}]$ is *nonnegative* if $a_{ij} \geq 0$ for each i and j. Matrices derived from data, or from real problems, are often nonnegative– the entries have some real interpretation. If we take the SVD factorisation $A = UV$ of such a matrix, we will often get entries with mixed signs, which can often make interpretation difficult.

Eigenfaces

Consider the example of facial feature recognition. Say that every column of A is a face– each entry encodes the grayscale of a pixel. In a good decomposition $A = UV$ of such a data matrix, we might hope that the columns of U, from which each face in A is build as a linear combination, are 'typical faces', or 'typical facial features'. such as an eye. So we want the columns of U to also be nonnegative. And we would like to build faces from the typical faces by adding them together. Subtracting a picture is difficult to interpret. So V should also be nonnegative. Our decomposition now builds faces from some basic parts in an understandable way.

That is an ideal. It practice, we cannot factor nonnegative A into nonnegative UV with orthogonal columns. Even giving up orthogonality, we cannot get a meaningful decomposition of a given A into nonnegative U and V. But are generally approximating A anyways, so we can build the nonnegativity of the factors into our approximation and instead of minimising $\|A - UV\|$ over orthogonal rank r matrices U and V^T, we will minimise $\|A - UV\|$ over nonnegative rank t matrices U and V^T.

In Chapter III we look at finding the rank k nonnegative matrices U and V that minimize

$$\|A - UV\|_F.$$

Sparseness of U and/or V

A matrix is *sparse* if many of its entries are 0. Using sparse matrices gives obvious computational advantages. And the facial features recognition lends to this in a natural way– a column of U that only deals with the eyes is going to be sparse.

In a way similar to how minimizing vector solutions with respect to the the ℓ_1-norm yields sparce vectors, minimising the nuclear norm $\|UV\|_N$ will tend to give sparse matrices. In Chapter IV and V we look at finding the rank k matrices that minimize

$$\|A - UV\|_F^2 + \lambda\|UV\|_N,$$

for some fixed weight λ.

I.12.2 Tensors

Whereas a matrix $M = [m_{ij}]$ is a 2-dimensional array of numbers, a d-way tensor is a d-dimensional array. We will mostly only talk of 3-way tensors, so a tensor is a 3-dimensional array $T = [T_{ijk}]$ of numbers for $i \in [I], j \in [J]$, and $k \in [K]$.

Note

The author mentions that a tensor is usually considered to have dimensions $m \times n \times p$, the m, n, and p refering to the rows, columns, and *tubes*, respectively; but he doesn't seem to use this notation.

A *slice* of T is a 2-way tensor, (or matrix), such as the $I \times J$ matrix T_{*j*} we get by fixing j, and we get a *fibre* by fixing two of the indices.

There are a couple of natural uses for tensors.

Example I.26. Where as a black and white photo might be encoded as a matrix of pixel intensities, in a colour photo the pixel has three (or four?) intensities, so naturally gives a tensor with $k = 3$ tubes.

Statistical samples often lead to tensors.

Example I.27 (Joint probablility tensor)**.** A population of children is sampled for membership into I age groups, J height groups, and K weight groups. With a $I \times J \times K$ tensor $P = [P_{ijk}]$ one holds the probabilities $p_{i,j,k}$ that a random child will be in age group i, height group j, and weight group k.

Entries of the tensor will add to 1. The entries of the slice P_{2**} sum to the probability that a random child is in the second age group.

I.12.3 The Norm and Rank of a Tensor

Our examples above are of 'data tensors'. Like matrices would could try to use tensors as operators that act on vectors or matrices. One can define products of tensors. But it seems to be difficult to generalise the theory of matrices to tensors in a meaningful way.

That said, one useful analogue is that of the outer product, it is used construct tensor, and so approximate them by low rank tensors.

A tensor T has *rank* 1 if there are vectors a, b and c such that

$$T = a \circ b \circ c \qquad \text{i.e. } T_{ijk} = a_i b_j c_k.$$

The *rank* of a tensor T is the minimum r such that T is the sum of rank 1 tensors.

There is an algorithm for finding the rank of a tensor, and one to find the rank r *CP-decomposition* of a tensor, which is the closest rank r approximation of a tensor. But finding the rank is an NP-hard problem, so both of these algorithms take exponential time.

We don't find the best approximation like the SVD does for matrices, but we can find a pretty good one. The idea is to slice the tensor into matrices and solve some matrix problems.

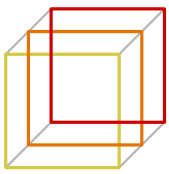
I.12.4 Unfolding a tensor

We want to approximate a tensor T by

$$T \approx \sum_{\alpha=1}^{r} A_\alpha \circ B_\alpha \circ C_\alpha,$$

where the A_α, B_α and C_α are vectors of lengths I, J and K respectively.

Assume that we actually have equality, and temporarily, that $r = 1$ so $T = a \circ b \circ c$, for column vectors a, b and c. We will first unfold T to a matrix T_1 that we get by concatenating the slices we get by fixing the index k:



The columns of this unfolded version T_1 of T are all scales of a. The scaling factors as we go along are

$$b_1c_1, b_2c_1, b_3c_1, \ldots, b_Jc_1, b_1c_2, \ldots, b_Jc_I =: b \oplus c.$$

So we write T_1 as the outerproduct $(b \oplus c)^T$. Similarily we unfold T in the other two directions:

$$T_1 = a(b \oplus c)^T \qquad \text{and} \qquad T_2 = b(c \oplus a)^T \qquad \text{and} \qquad T_3 = c(a \oplus b).$$

(This $\oplus$ is a special case of what is known as the Kronecker product for matrices.)

When T has rank r we build matrices A, B and C from the column vectors A_α, B_α and C_α and sum the T_1 for each vector into a rank r version of T_1.

For this we define the Khatri-Rao product $B \odot C$ for these r column matrices as the r column matrix $B \odot C$ whose α^{th} column is $B_\alpha \oplus C_\alpha$. Thus

$$T_1 = A(B \odot C)^T \qquad \text{and} \qquad T_2 = B(C \oplus A)^T \qquad \text{and} \qquad T_3 = C(A \oplus B).$$

To approximate T by a rank r tensor, we will slice T into T_1, T_2 and T_3, and then find A, B and C that minimise

$$\|T_1 - A(B \odot C)^T\|_F^2$$

for fixed B and C, by least squares. Then we fix A and C and improve B in the same way, and then improve C. We then iterate, depending on how much we want to improve the approximation.

Problems from the text

I.12:

II Computations with Large Matrices

We have seen how to

- solve the matrix equation $Ax = b$,
- find an orthoganal basis for $C(A)$ (Gram Schmidt)
- project vectors onto $C(A)$ to get the 'least squares' solution to $Ax = b$, (Compute projection matrix $P_A = A(A^TA)^{-1}A^T$),
- find the eigens of A, or its more generally its singular value decomposition to approximate it by a low rank matrix.

These are all skills we want, but we can really only do it for small matrices. For large matrices, Gaussian Elimination takes too long, and finding eigenvalues algebraically becomes impossible.

In this section we look at doing these things numerically. This is how computers are going to do them, so we should understand this.

Our solutions will not be exact. There will alway be round-off error even in the representation of matrices. So we will use norms extensively to understand how good our approximations are, and to guide us in iterative computations.

The condition number of an invertible matrix

When we want to solve the equation $Ax = b$ on a computer, round-off (or other things) can introduce error to both of A and b and this will introduce error to our solution. It is good to know how much error ε_x we can get in x from some small error ε_b in b or ε_A in A. Generally we want to consider relative error: how can $\frac{|\varepsilon_x|}{|x|}$ grow compared to $\frac{|\varepsilon_b|}{|b|}$?

Well, solving $Ax = b + \varepsilon_b$ we get a solution $x + \varepsilon_x$ such that

$$b + \varepsilon_b = A(x + \varepsilon_x) = Ax + A\varepsilon_x$$

so $\varepsilon_b = A\varepsilon_x$, and $|\varepsilon_b| \leq \|A\||\varepsilon_b|$. As $x = A^{-1}b$ (as A is invertible) we also have $|x| \leq \|A^{-1}\||b|$, and so get

$$\frac{|\varepsilon_b|}{|b|} \leq \|A\|\|A^{-1}\|\frac{|\varepsilon_x|}{|x|}.$$

This $c = \|A\|\|A^{-1}\|$ is the *condition number* of A.

Problem

Show that the condition number of A is $\frac{|\lambda_M|}{|\lambda_m|}$ these are, by absolute value, the largest and smallest eigenvalues of A. (Hint: if λ is an eigenvalue of A, what eigenvalue of A^{-1} do you know.)

A small condition number for A tells us that a small change in b only gives a small change in the solution x to $Ax = b$. One can similarily show that it ensures a small error in A only gives a small change in x (though relative change is measured slightly differently).

II.1 Numerical Linear Algebra

We are going to use a lot of iterative computations to speed up our operations. As a simple introduction, we first look at solving $Ax = b$ by splitting A; by this, we mean we write $A = S - T$ for matrices S and T that we will choose later. Thus we can write $Ax = b$ as

$$Sx = Tx + b.$$

Now starting with a guessed solution x_0 at x we solve

$$Sx_1 = Tx_0 + b.$$

Consider now the error of these solutions compared to the real solution x. We have

$$S(x - x_1) = T(x - x_0) \qquad \text{and so} \qquad (x - x_1) = S^{-1}T(x - x_0).$$

If we can ensure that $S^{-1}T$ has largest eigenvalue $\lambda_M < 1$ then this reduces the error by at least λ_M. The smaller the eigenvalue, the faster the convergence of the iterated solutions. The method of conjugate gradients will do this wonderfully when A is symmetric. It will take a couple of steps to get there, but all steps are nice ideas. As we are talking about eigenvalues, A is an $n \times n$ matrix for the rest of this section.

II.1.1 The QR algorithm for computing eigenvalues

Recall that Gram-Schmidt allows us to find an orthogonal basis $\{q_1, \ldots, q_n\}$ for the column space of A. The orthogonal basis is nice because it is easy to write a vector in the space as a linear combination of basis vector:

$$v = (q_1^T v)q_1 + (q_2^T v)q_2 + \cdots + (q_n^T v)q_n$$

and in particular, as we got these vectors from $a_1, \ldots, a_n$ via Gram Schmidt, we get

$$a_i = (q_1^T v)q_1 + \cdots + (q_i^T v)q_i + 0 + 0 + \cdots + 0.$$

This gives us the QR-decomposition of A

$$\underbrace{\begin{bmatrix} a_1 \mid a_2 \mid \cdots \mid a_n \end{bmatrix}}_{A} = \underbrace{\begin{bmatrix} q_1 \mid q_2 \mid \cdots \mid q_n \end{bmatrix}}_{Q} \underbrace{\begin{bmatrix} q_1^T a_1 & q_1^T a_2 & \ldots & q_1^T a_n \\ 0 & q_2^T a_2 & \ldots & q_2^T a_n \\ & & \ddots & \\ 0 & 0 & \ldots & q_2^T a_n \end{bmatrix}}_{R}$$

of A where R is upper triangular. This takes $O(n^3)$ basic operations.

The QR iteration algorithm for a matrix $A =: A_0$ first finds its QR-decompositon $A_0 = Q_0 R_0$ and then sets $A_1 = R_0 Q_0$. Thus $A_1 = Q_0^T Q_0 R_0 Q_0 = Q_0 T A_0 Q_0^T$, and so A_1 and A_0 are similar. Thus they have the same eigenvalues. Iterating one computes $A_2, A_3, \ldots$, that all have the same eigenvalues as A_0.

What makes this useful then is that the matrices A_i usually tend to diagonal, the values furthest away from the diagonal tend to shrink the fastest, and so the diagonal entries tend to the eigenvalues of A. Generally what happens is that the last diagonal entry (the (n, n) entry) of A_i first approaches the smallest (in absolute value) eigenvalue. When it seems to have converged enough, when the rest of the entries in its row and column are small enough, we rip them off and work with the smaller matrix.

This can take a while though. There are a couple of improvement that speed things up a lot: shifting and tridiagonalistion.

Say we have a good approximation s to an eigenvalue λ of A; perhaps we take the (n, n) entry of A_i. Then $A - \lambda I$ has an eigenvalue of 0 and $A_i - sI$ has an eigenvalue close to 0. Taking

$$A_{i+1} = R_i Q_i + sI$$

we get a matrix that is similar to A_i, and so has the same eigenvalues. A couple of iterations, and this tiny eigenvalue will show up on the last diagonal. This tends to work quite well.

Another speed up is to start with a matrix, similar to A, that is already close to diagonal– then iteration is way faster. The rest of the section focusses on finding a triadiagonal matrix that is similar to symmetric A.

Now that we know the eigenvalues, we can get the eigenvectors of A too. Let the eigenvectors be $\lambda_1, \ldots, \lambda_n$ from smallest absolute value to largest, and $x_1, \ldots x_n$ be the corresponding eigenvectors. A random vector v will almost always be of the form

$$v = \sum_{i=1}^{n} c_i x_i$$

for non-zero c_i. Scaling Ax we get that

$$(Av)/\lambda_n = \sum_{i=1}^{n} \frac{\lambda_i}{\lambda_n} c_i x_i.$$

and $A^n v/\lambda_n \to c_n x_n$.

II.1.2 Krylov Subspaces and Arnoldi Iteration

Given a full rank square matrix A, Krylov defined a series of subspaces that can be used quite effectively in approximating, and eventually solving, the equation $Ax = b$. Recall that Gram Schmidt allows us to find an orthogonal basis $q_1, \ldots, q_n$ for the column space of A.

Krylov used Gram Schmidt not on the columns, but some other vectors a_i. The k^{th} *Krylov subspace* of $C(A)$ for $k = 1, \ldots, r$ is

$$K_k = \langle b, Ab, A^2 b, \ldots, A^{k-1} b \rangle.$$

It is a subspace of $C(A)$, as all generators are in $C(A)$, usually has rank k, and can be computed with k Matrix-vector products, so in time $O(k \cdot n^2)$.

Orthogonalising the generators of K_k with Gram-Schmidt leads to quite a useful matrix

$$H_k = Q_k^T A Q_k$$

where the columns of Q_k are the orthoganal basis, and H_k is what is called *Hesseberg*– it has zeros below the first subdiagonal.

Indeed, we compute H_k using Arnoldi's tiny variation on Gram-Schmidt.

Gram–Schmidt	**Arnoldi**
$q_1 = \dfrac{b}{\|b\|}$	$q_1 = \dfrac{b}{\|b\|}$
$q'_{k+1} = A^k b - \sum_{i=1}^{k} (q_i^T A^k b) q_i$	$q'_{k+1} = Aq_k - \sum_{i=1}^{k} (q_i^T A q_k) q_i$
$q_{k+1} = \dfrac{q'_{k+1}}{\|q'_{k+1}\|}$	$q_{k+1} = \dfrac{q'_{k+1}}{\|q'_{k+1}\|}$

Notice that both algorithms give the same q_{k+1}. Indeed q_k is $A^{k-1}b$ with some projections onto K_{k-1} removed; multiplying by A takes the removed bit into K_k, so it is removed anyway.

Now rewriting the calculation of q_{k+1} with column vectors and matrices, we get

$$\|q'_{k+1}\| \begin{bmatrix} | \\ q_{k+1} \\ | \end{bmatrix} = \begin{bmatrix} | \\ q'_{k+1} \\ | \end{bmatrix} = Aq_k - \begin{bmatrix} q_1 | q_2 | \dots | q_k \end{bmatrix} \begin{bmatrix} q_1^T(Aq_k) \\ q_2^T(Aq_k) \\ \vdots \\ q_k^T(Aq_k) \end{bmatrix}.$$

Rearranging things this gives

$$Aq_k = \begin{bmatrix} q_1 | q_2 | \dots | q_{k+1} \end{bmatrix} \begin{bmatrix} q_1^T(Aq_k) \\ q_2^T(Aq_k) \\ \vdots \\ q_k^T(Aq_k) \\ \|q'_{k+1}\| \end{bmatrix}$$

and putting this together for all k give

$$A \begin{bmatrix} q_1 | q_2 | \dots | q_k \end{bmatrix} = \begin{bmatrix} q_1 | q_2 | \dots | q_{k+1} \end{bmatrix} \begin{bmatrix} h_{11} & \dots & h_{1k} \\ h_{21} & \dots & \vdots \\ & \ddots & h_{k+1,k} \end{bmatrix}$$

where $h_{i,k} = q_i^T(Aq_k)$ for $i = 1, \dots, k$ and $h_{i+1,i}$ is $\|q'_i\|$.

Interpretation of $H_k = Q_k^T A Q_k$

Strang says "H_k is the projection of A onto K_k using the basis of qs." I found this confusing, because H_k clearly is not the projection matrix for K_k– that is $P = Q_k Q_k^T$. So what does he mean. I think it is this.

The $n \times k$ matrix Q_k bijects the vectors of $\mathbb{R}^k$ to the space $K_k \subset \mathbb{R}^n$, and Q_k^T takes them back. The matrix Q_k^T is called a *compression* of K_k to $\mathbb{R}^k$. It allows us to work with K_k using vectors of size k rather than of size k. Notices that it also works as a projection– vectors in K_k map to $\mathbb{R}^k$, but for vectors x of $\mathbb{R}^n$ that are orthogonal to K_k we have that $Q_k^T x = 0$. We view the action of A on K_k as being done by PA. Okay, but what does this look like in the compressed picture of K_k in $\mathbb{R}^k$? Well $H_k = Q_k^T A Q_k$ tells us this; the following diagram commutes:

$$\begin{array}{ccc} \mathcal{K}_k & \xrightarrow{Q_k Q_k^T A} & \mathcal{K}_k \\ \downarrow Q_k^T & & \downarrow Q_k^T \\ \mathbb{R}^k & \xrightarrow{H_k} & \mathbb{R}^k \end{array}$$

So H_k is the action of A projected onto K_k, (and then compressed down to $\mathbb{R}^k$) by the basis Q_k.

We note now, that when $k = n$, H_k is similar to A, so has the same eigenvectors. When $k < n$ the matrices are not similar, but often, the eigenvalues of H_k are pretty close to the k largest eigenvalues of A. (We are reminded that this is only 'often', not always.)

II.1.3 Symmetric Matrices: Arnoldi Becomes Lanczos

Computing with $H_k = Q_k^T A Q_k$ is nice, being $k \times k$ rather than $n \times n$, but also because it is almost upper triangular. When A is symmetric, then so is H_k, and so it is *tridiagonal*– also zero above the first superdiagonal, so computing is even faster. In this situation, where A is a symmetric matrix S, we write H_k and T_k and so write

$$T_k = Q_k^T S Q_k.$$

Computing T_k is much facter too– a lot of the projections $h_{i,j}$ that we had to computer to compute H_k are automatically 0.

II.1.4 Computing eigenvalues of T by QR iteration

Putting everything together, one of the fastest ways to compute the eigenvalues of a symmetric matrix S is to

i. Tridiagonalise $T = Q^T A Q$ using Lanczos (with $k = n$).

ii. Set $T_0 = T$ and use shifted QR-iteration.

On a tridiagonal T, the shifted QR-iteration takes $O(n^3/\varepsilon)$ time for multiplicative error of at most ε in the eigenvalues.

II.1.5 Computing the SVD

Okay. We have found the eigenvalues and eigenvectors of symmetric S. To find the SVD $A = U\Sigma V^T$ of A we can find the singular values and vectors by finding the eigenvalues and vectors of A^TA and AA^T.

Well if $A_* = Q_1AQ_2^T$ for orthogonal matrices Q_1 and Q_2, then it is easy to show that $A_*^TA_*$ is similar to A^TA, and $A_*A_*^T$ to AA^T, so A and A' have the same singulars. As Q_1 and Q_2 don't have to be the same, there is more freedom in sparsifying A, and one can actually use Gram Schmidt to find Q_1 and Q_2 so that $Q_1AQ_2^T$ is bidiagonal. One can then vary the shifted QR algorithm a bit to compute the singular values of the bidiagonal matrix. This is described in detail in Golub-Van Loan.

II.1.6 Conjugate Gradients for $Sx = b$

To find the solution x_* to $Sx = b$, we will iterate using the S-norm $\|u, v\|_S = u^TSv$. We say that u and v are *conjugate* if $u^TSv = 0$. One can show that a set $\{d_1, \ldots, d_n\}$ of n mutually conjugate vectors are a basis of $\mathbb{R}^n$, so our solution can be written as

$$x_* = \sum_{i=1}^{n} \alpha_i d_i.$$

If we can find such a set $\{d_1, \ldots, d_n\}$ then can compute x_* by computing each of the the α_i:

$$d_i^Tb = d_i^TSx_* = d_i^TS\sum_{i=1}^{n} \alpha_i d_i = \alpha_i d_i^TAd_i.$$

We don't want to have to compute all the α_i though, so if we can find a nice set $\{d_1, \ldots, d_n\}$ we can get close enough to the solution by just computing some of them.

There is a cute trick for this. The function $Sx - b$ is the gradient ∇_f of the paraboloid

$$f(x) = \frac{1}{2}x^TSx - x^Tb.$$

Recall that, in general, $\nabla_f(x)$ is a vector in the direction of fastest increase of f at x, and its norm is how fast we increase went moving x a distance of 1 in that direction. When $Sx - b = \nabla_f(x) = 0$, the point x is a critical point of f, and so a minimum when S is positive definite. Thus to find x_* is it enough to minimize $f(x)$.

Guessing a solution x_0 we compute the gradient $\nabla_f(x_0) = Sx_0 - b$. We want to decrease f, not increase so we will move in the direction of the *residual* $r_0 = b - Sx_0$. We take this as our first basis vector: $d_0 = r_0$. Having determined the direction d_0 we will move, and knowing approximately the distance $r_0^Tr_0$ that we want to move, we normalise, and add $\frac{r_0^tr_0}{d_0^TSd_0}x_0 =: \alpha_0x_0$ to our solution: $x_1 = x_0 + \alpha_0d_0$.

Then iterate:

i. Step size $\alpha_{k-1} = \frac{r_{k-1}^t r_{k-1}}{d_{k-1}^T S d_{k-1}}$

ii. Update solution $x_k = x_{k-1} + \alpha_{k-1} d_{k-1}$

iii. Residual $r_k = b - \alpha_k S d_{k-1}$

iv. Next basis vector $d_k = r_k - \sum \text{proj}_{r_i} r_k$

When the residual gets small enough, we can stop. The conjugation of the basis by the Gram Schmidt type step can be speed up using Krylov spaces so the computation of r_k in the book is $r_k = r_{k-1} - \alpha_k S d_{k-1}$. And the basis vector, instead, is $d_k = r_k + \frac{r_k^T r_k}{r_{k-1}^T r_{k-1}}$. I think this compensates. I am not quite sure, I couldn't figure it out in time. But this seems to be the version that is used.

II.2 Least Squares: Four ways

In this section we look at how to solve a least squares problem quickly and accurately.

The least squares problem is to find a best approximation $\hat{x}$ to an equation

$$Ax = b$$

that has no solution; this is generally only issue for thin A: $n \leq m$.

We found $\hat{x}$ by projecting b to $\hat{b}$ in $C(A)$

$$\hat{b} = P_A b = A(A^T A)^{-1} A^T b$$

and finding the solution to $Ax = \hat{b}$.

Practice

Usually least squares is done when A is thin: $n < m$. With this in mind, decide which multiplications are best done first in computing $\hat{b}$? How long does it take to compute $\hat{x}$ by basic methods (Gaussian eliminate for inverting $A^T A$ and for solving $Ax = \hat{b}$?)

The above producure for computing $\hat{x}$ only works if $A^T A$ is invertible.

Practice

Show that the following are equivalent for a matrix A.

i. The columns of A are independent.

ii. $A^T A$ is invertible.

iii. $Ax = \hat{b}$, where $\hat{b}$ is the projection of b onto $C(A)$, has a unique solution for all b.

A lot of the techniques for finding the least square approximation of $Ax = b$ assume that A has independent columns.

Strang gives 'Four ways' to compute $\hat{x}$.

Assuming it does, then $\hat{x} = (A^TA)^{-1}A^Tb$ is certainly a solution to $Ax = \hat{b}$ and so (by the exercise) is the unique solution.

When A has independent columns, the matrix $A^+ := (A^TA)^{-1}A^T$ acts like an inverse with repect to the least squares approximation– to find the solution to $Ax = \hat{b}$ we mulpily b by A^+. Indeed, $x^+ = A^+b$ is the solution $\hat{x}$ as

$$Ax^+ = AA^+b = A(A^TA)^{-1}A^Tb = P_Ab = \hat{b}.$$

Moreover, if A is invertible we have that $A^+ = (A^TA)^{-1}A^T = A^{-1}(A^T)^{-1}A^T = A^{-1}$. When A is thin, $A^+ := (A^TA)^{-1}A^T$ is the *pseudo-inverse* of A.

When there is not a unique solution $\hat{x}$, to $A\hat{x} = \hat{b}$, one not only tries to find one solution, but the best one. We will give a definition the pseudo-inverse A^+ that works in this case, and $x^+ = A^+b$ will be the solution with minimum norm; it is called the *minimum norm least squares solution.*

This is Strang's first of four ways of solving least squares. We will look at how the pseudo-inverse A^+ is defined when A is fat, and see how it comes from the SVD of A. Whether A has independent columns or not, $x^+ = A^+b$ will be the minimum norm least squares solution of $Ax = b$.

The second of four ways is to solve the normal equation

$$A^TA\hat{x} = A^Tb.$$

This saves inverting A^TA, but still only works when it is invertible.

The third of four ways is to use a QR-decomposition $A = QR$ of A. Doing so gives

$$A^TA = (QR)^TQR = R^TQ^TQR = R^TR$$

which speeds up computations in the preceeding two methods. Furthermore, finding the decomposition with Gram-Schmidt effectively removes dependent columns of A as we go, so saves us having to deal with this before we get started. We will look at several computational ideas that help when doing Gram-Schmidt.

Strang's fourth of four ways to find x^+ is to add small perturbations of norm δ to A, making the columns independent, and finding an approximation $\hat{x}_\delta$ to x^+. Letting δ go to 0, this goes to x^+.

So: pseudo-inverse, then Gram-schmidt, then perturbations.

II.2.1 The Pseudo-inverse

The *pseudo inverse* of a matrix A with SVD $A = U\Sigma V^T$ is $A^+ = V\Sigma^+U^T$ where the pseudo inverse Σ^+ of an $m \times n$ matrix Σ that is 0 on non-diagonal entries is the $n \times m$ matrix we get from Σ by transposing it and reciprocating non-zero entries.

The defining of Σ^+ is so that $\Sigma^+\Sigma$ and $\Sigma\Sigma^+$ are square matrices with an I in an appropriately sized upper-left block, and 0 elsewhere.

Practice

Show that when A has independent columns, $A^+A = I$ and $A^+ = (A^TA)^{-1}A^T$. Show that when A has independent rows $AA^+ = I$ and $A^+ = A^T(AA^T)^{-1}$.

When A doesn't have independent columns then there are many solutions $\hat{x}$ to

$$Ax = \hat{b}.$$

Moreover, as $A^+ = V\Sigma^+U^T$ we can get other solutions by adding vectors in the nullspace of A. This corresponds to changing entries in x^+ that correspond to the columns of Σ^+ with zero entries. The value of $\|x^+\|$ is minimum when these are 0, which is what we get from $x^+ = A^+b$. So x^+ is the minimum norm solution to $Ax = b$.

Take-away: $x^+ = A^+b$ is the least squares solution to $Ax = b$ when A has independent columns, and the minimum norm solution when A has independent rows. In general we may have neither, but x^+ is still the the minimum norm least squares approximation. We know how to compute it because we can compute the SVD of A.

II.2.2 Gram Schmidt and QR-decompositions

We have seen the Gram Schmidt algorithm for orthogonalising the columns of a matrix A, and so getting a decompositon $A = QR$:

$$\underbrace{\begin{bmatrix} a_1 \mid a_2 \mid \cdots \mid a_n \end{bmatrix}}_{A} = \underbrace{\begin{bmatrix} q_1 \mid q_2 \mid \cdots \mid q_n \end{bmatrix}}_{Q} \underbrace{\begin{bmatrix} r_{1,1} & r_{1,2} & \dots & r_{1,n} \\ 0 & r_{2,2} & \dots & r_{2,n} \\ & & \ddots & \\ 0 & 0 & \dots & r_{n,n} \end{bmatrix}}_{R}$$

Algorithm II.1. *Gram-Schmidt for $A = A_{m\times n}$ to produce $A = QR$.*

For $i = 1, \dots n$ do:

i. Compute $q_i' = a_i - \sum_{j=1}^{i-1} r_{ji}q_j$ where $r_{ji} = q_j^T a_i$.

ii. Normalise $q_i = q_i'/r_{ii}$ where $r_{ii} = \|q_i'\| (= q_i^T a_i)$.

When we have error in our numbers, introduced by computer round-off or measurement uncertainty, this error will start to propegate through this process, and we have to be aware of it. More importantly, say we have two similar columns a_1 and a_2 of A such that most of a_2 lives in the projection of a_1. The error in a_2 tends to be random, so when we remove the projection onto a_1 from a_2, it is getting small, but the error is does not shrink so much. So the residual a_2 can end up consisting mostly of error, and this will propegate to devestating effect.

To avoid this, we remove the a_1 component from all remaining vectors, and then move next to the one with largest residual– it should have relatively less error.

In practice, Gram Schmidt yields less error when we reorder the columns of A as we go according to their residuals.

This yields a decomposition

$$AP = QR$$

where P is a permutation matrix.

Algorithm II.2. *Gram-Schmidt with pivoting for $A = A_{m\times n}$ to produce $A = QR$.*

Start with $I = P$ and for $i = 1, \dots n$ do:

i. Normalise $q_i = q_i'/r_{ii}$ where $r_{ii} = \|q_i'\| (= q_i^T a_i)$.

ii. For $j = i+1, \dots, n$ update $q_j' = q_j' - r_{ij}$ where $r_{ij} = q_i^T q_j' = q_i^T a_j$.

iii. Choose the $q_m' \in \{q_{i+1}', \dots, q_n'\}$ with max norm, switch it with q_{i+1}', and set $q_{i+1} = q_{i+1}'$. (Switch columns $i+1$ and m of P.)

Overall, we were doing $O(n^2)$ projections then subtractions of vectors in $\mathbb{R}^m$, so the non-pivoting algorithm is $O(mn^2)$. To each of these we add the computation of a norm. This adds another $O(mn^2)$. Finding the min of list of n elements takes $O(n)$ so this adds $O(n^2)$ to the algorithm. So the algorithm is still $O(mn^2)$. We've multiplied the running time by about 3/2.

Practice

Show how, by keeping track of the squared norm of vectors using $O(n)$ space, the operation update the norm when we update the vector requires only a single operation, adding time $O(mn)$ over the whole algorithm rather than $O(mn^2)$.

Another step suggested in the text, "for safety" is *re-othanormalising*. In computing q_i we have removed the projections onto the $q_1, \dots q_{i-1}$. As there is error, removing the projection onto q_3 may add some error back in the q_1 direction. So it seems effective to reorthanormlise q_i after computing it by again removing the projections onto the earlier q_j, and normalising again. (Presumably the error added the second time is less. Can we prove this?)

Practice: Computational Project

Use Gram Schmidt to orthogonalise $\begin{bmatrix} 1 & 7 & 13 \\ 3 & 41 & 6 \\ 5 & 10 & 2 \end{bmatrix}$ (or an even bigger random matrix) and find $A = QR$.
Do it on a computer with real numbers to two decimal places so that there is round-off error. Then use the suggested algorithm to find $AP = QR$.
aa Find the approximate error of the columns of Q for the two decompositions. (How? Perhaps redo the calculation with real numbers with more decimal places to get decompositions that are closer to correct. Or do them algebraically and then evaluate entries to some appropriate number of decimal places.
Try again with re-orthanormalisation.

II.2.3 Householder Reflections

The text outlines another common technique for finding the QR decomposition of a matrix A that is computationally effective.

II.2.4 Ridge regression method for computing the pseudo inverse

The SVD method seems to be the faster method used to get A^+, but when A^TA has singular values that are zero, A^+ is unstable, a small change in A that changes the singular value to non-zero can radically change A^+. As we are probably allowing some error in A, we can introduce some to keep the singular values of A^TA away from zero, and so get a stable pseudo inverse of the slightly wrong A. This doesn't use the SVD of A.

We saw that when A^TA is invertible, then it is $A^+ = (A^TA)^{-1}A^T$. As A^TA is symmetric positive semidefinate, we can make it invertible by adding a little bit, δI, to the diagonal.

We have that

$$A^TA + \delta I = V\Sigma^TU^TU\Sigma V^T + V\delta IV^T = V(\Sigma^T\Sigma + \delta I)V^T$$

and so

$$(A^TA + \delta I)^{-1}A^T = V(\Sigma^T\Sigma + \delta I)V^TV\Sigma^TU^T = V(\Sigma^T\Sigma + \delta I)\Sigma^T).$$

The U and V are independent of δ! As δ goes to zero, we get A^+. But if we suspect that A has a singular value of 0, it is best to just take δ small but non-zero, and compute $A^+ \sim (A^TA+\delta I)^{-1}A^T$.

II.3 Three bases for $C(A)$

In this chapter Strang re-iterates that an $m \times n$ matrix of low rank $r < \min m, n$ can often be made to have higher rank by the addition of some small random noise, and that one goal is to de-couple the noise, writing it as $A + E$ where A has smaller *effective* rank, and the noise matrix E has high rank, but small norm.

We got a low-rank approximation of A from an SVD $A = U\Sigma V^T$. Ordering the columns so that the entries of Σ were non-increasing in absolute value as we went down the diagonal, we said that

$$A_r = \sum_{i=1}^{r} u_i\sigma_i v_i^T = U_r\Sigma_r V_r^T$$

was the best rank r approximation of A in that it minimised $\|E\|$.

But other rank r approximations of A (coming from other factorisations) have their own benefits. One of these factorisations is $A = QR$, and the other comes in a couple of forms: we will see $A = CMR$ and $A = YBZ$. These will have the benefit that columns of C are columns of A, rows of R are rows of A, and B is a square submatrix of A. This is often useful in that these matrices then preserve the interpretation of rows and columns of A, as can be desirable if A is a data matrix.

(In a factorisation like $A = U\Sigma V^T$, the columns of U are a basis of $C(A)$. Strang's 'three bases' is referring to the bases we get from various factorisations of A.)

The main idea in this chapter is that we do not need to get the whole factorisation– reordering columns as we go, we can keep only the 'best' columns in $C(A)$. Generally we do not have a good measure of the effective rank r, so we do not know how many of the columns we will keep, but we can often have a allowed amound of error $\varepsilon = \|E\|$, and we take columns until we are below it.

II.3.1 Partial Gram Schmidt with pivoting

The simplest version of this comes from Gram Schmidt with pivoting, which we saw in the previous section. To choose the new column q'_{i+1} we have already computed the norms of the remaining columns $q'_{i+2} \dots q'_n$. We have a decomposition

$$A = [\, Q_r R_r P^T \,|\, A_{n-r} P^T]$$

where P is simply permuting columns, $Q_r R_r$ is the QR decomposition of the first r columns of (column-permuted) A, and A_{n-r} is the last $n-r$ columns of (column-permuted) A. As we have the norms of the columns of A_{n-r} we have its norm, and stop when this norm is below the acceptable error ε.

II.3.2 SVD from Partial Gram Schmidt

Once we've used partial Gram Schmidt to reduce A to $A = Q_r R_r P^T + E$ for small norm error matrix E we can then find a small SVD of $R_r P^T$ which has r rows:

$$R_r P^T = U_r \Sigma V^T.$$

As Q_r and U_r are both orthogonal, so is their product $U = Q_r U_r$, and so we get an approximate SVD

$$A = Q_r R_r P^T + E = Q_r U_r \Sigma V^T + E = U \Sigma C V^T + E$$

of A.

II.3.3 Interpolative Decomposition

As mentioned above, a clever student can see why we might want to use a basis of columns of A to give a low rank approximation of A. There are a couple of models for this, called interpolative decompositions, ID, for short. These assume a rank of r for A, and if this is true, then they are decompositions. If the rank is higher than r, then they are just rank r approximations.

YBZ

The first one decomposes A into $A = YBZ$ where B is an $r \times r$ square submatrix of A.

Using Gaussian elimination, we can decompose

$$A = CZ$$

where $C = C_{m\times r}$ consists of columns of A that are a basis of $C(A)$. This C has rank r, so we can decompose it into

$$C = YB$$

by finding a basis of its rowspace, and making these the rows of B. (We can do this, say, with Gaussian elimination on C^T.)

Then $A = YBZ$ as desired. By permuting rows and columns, we may assume that B was the $r \times r$ matrix in the upper left corner of A and so our matrices have the form

$$A = YBZ = \left[\begin{array}{c} I_r \\ \hline C_{m-r}B^{-1} \end{array}\right]_{m\times r} \left[\, B \,\right]_{r\times r} \left[\, I_r \mid B^{-1}Z_{n-r} \,\right]_{r\times n}$$

where C_{m-n} is the last $m-r$ columns of A and Z_{n-r} is the last $n-r$ rows of Z.

The goal is to select the columns and rows of A that make up B so that $C_{m-r}B^{-1}$ and $B^{-1}Z_{n-r}$ have small entries. It is possible to choose them so that all entries in $C_{m-r}B^{-1}$ and $B^{-1}Z_{n-r}$ are less than 1. Cramer's rule tells us that the i^{th} entry x_i of the solution x of an equation

$$Ax = b$$

can be computed as $x_i = \frac{\det A*}{\det A}$ where we get A^* from A by replacing its i^{th} column by b.

Restricting the equation $B(B^{-1}Z_{n-r}) = Z_{n-r}$ to the j^{th} column we get that the ij^{ht} entry of $B^{-1}Z_{n-r}$ is $\frac{\det B^*}{\det B}$. We can choose the columns of A that go into B to maximise its determinant, and so get that this is less than 1 for all entries.

While this is possible, it takes a long time. So apparently one instead uses a random algorithm, which, whp, ensures the entries are less than 2. (It is not clear yet whether the details of this are given in the text.)

CMR

The other ID of A decomposes it into $A = CMR$ where $C = C_{m\times r}$ contains r columns of A, $R = R_{r\times n}$ contains r rows of A, and M is called a mixing matrix. Choosing any r independent columns of A for columns of C and r independent rows of A for rows of R we can compute the mixing matrix M. Indeed, if C and R were invertible, we would have $M = C^{-1}AR^{-1}$. As they aren't in general invertible we can use the left inverse $(C^TC)^{-1}C^T$ of C and the right inverse $R^T(RR^T)^{-1}$ of R to write

$$M = (C^TC)^{-1}C^TAR^T(RR^T)^{-1}.$$

When we are using this to approximate A, the goal is to choose 'good' columns and rows of A for C and R to make a good approximation. To make $\|CMR - A\|$ small.

Oddly, there is a pretty good way to do this starting from a rank r SVD approximation of A

$$A \approx U_{m\times r}\Sigma V^T_{r\times n}.$$

To choose the rows of A to put in R we use $U = U_{m\times r}$. Do Gaussian elimination on U, but each time you eliminate below a pivot, reorder the rows not above the pivot to maximize the pivot. This gives an re-ordering of the rows of U to $P_R U$. Apply this reordering to A, getting $P_R A$, and then take the first r rows for R.

Do the same to V to get a reordering $V^T P_C$ of its columns, and take the first r columns of AP_C as C. Apparently the difference between the *interpolary projections* $C^T C^1 C^T A$ and $AR^T(RR^T)^{-1}$ of A, from A, are bounded by a 'modest' function of the $r+1$ singular value of A. So this controls how good the approximation is.

The details of this are not given in the text, but are in a paper called *A DEIM induced CUR factorisation* by Sorensen and Embree.

II.4 Randomized Linear Algebra

We start with a simple little example that gives a feel for the utility of randomized linear algebra. We then show how we can get various approximate decompositions of a matrix A with simple column sampling ideas. We were promised a random algorithm for the decomposition $A = YBZ$, but it doesn't appear in this chapter. There seems to be a new promise of a random algorithm for a $A = CZ$ decomposition (presumably this is the first step for the YBZ algorithm) but this is a vague Strangian promise, so who knows. We do however see how to use random techniqes to get an approximate QR decomposition of a matrix A, and so through this an approximate SVD decomposition.

II.4.1 Random Factorisations

To finish this brief In the rest of the chapter, we give randomised rank r approximate decompositions of an $m \times n$ matrix A with rank greater than r.

Starting with an $n \times r$ matrix G whose entries are random samples from the normal distribution $N(0,1)$ we compute

$$Y = AG.$$

This matrix has two important properties: with high probability, has rank r (as long as A had rank at least r), its columns generate a space that is very close to space generated by U_r from the rank r Eckart-Young approximation $A_k = U_k \Sigma_k V_K^T$ of A. To see the latter point, observe that columns of G are random vectors in $C(A)$. They tend to have random coefficients when we write them as a linear combination of the right singular vectors of A (columns of V^T). Acting on them by A we get columns of G that when written as linear combinations of the left singular vectors of A (columns of U) the vectors corresponding to the larger singular value are emphasised.

So G has rank r and its columns are skewed towards $C(U_k)$– it generates something close to $C(U_k)$.

So $P_Y = YY^+$ projects onto this space. It works almost like an identity on vectors in U_k. Now A should stretch these vectors, and it still will, and so multiplying them by YY^+A is basically the

same a mutliplying them by A. Thus

$$YY^+A \approx A_k \approx A.$$

Taking a QR decomposition of YY^+A gives an approximate QR decompositon of A, and taking an SVD decomposition gives an approximate SVD decomposition of A. But for the SVD decomposition we can get there quicker.

Taking a QR decomposition $Y = Q_Y R_Y$ gives an othogonal basis Q_Y of $C(Y)$ so $YY^+ = P_Y = Q_Y Q_Y^T$. And taking an SVD decompostion $Q_Y^T A = U\Sigma V^T$ we can then compute

$$A \approx YY^+A = Q_Y Q_Y^T A = QU\Sigma V^T.$$

The matrix QU is orthogonal, so this is our SVD approximation.

3 Conditioning and Stability

We have not really covered the notions of Conditioning and Stability, and the ideas of matrix calculus that are used for them enough to really understand the next sections of Strang. So this follows chapter from Trefethen and Bau's Text "Numerical Linear Algebra" that covers these ideas is more detail.

3.1 Conditioning and Condition numbers

We have talked about norm

$$\|A\| = \max_{x \in \mathbb{R}_n} \frac{\|Ax\|}{\|x\|}$$

of a matrix A, and the condition number

$$c = \|A\|\|A^{-1}\| = \frac{\lambda_M}{\lambda_m}$$

of an invertible matrix A.

The condition number was a measure of how much a change in b or A coud change the solution x of the problem $Ax = b$.

This is just a special case of the notion of the condition of a problem. More generally we might ask how a change in A affects the eigenvalues, or the determinants of A, or eigenvectors, or inverse of A. To do this in generality we describe any of these problems as a function $f : X \to Y$ between vector spaces.

Example 3.1. Solving $Ax = b$ can be viewed as a function $f : \mathbb{R}^{n^2} \to \mathbb{R}^n$ that such that for matrix A, $f(A) = A^{-1}b$ is the solution to $Ax = b$. Or it could be a function of b, or of A and b.

The condition of the problem $f(x)$ is then a measure of how much of a change in $f(x)$ we get from a small change in the x. This sounds like derivatives, and we use calculus to formalise this, so the notation will resemble calculus.

For a problem $f : X \to Y$ and $x \in X$, let u be some unit error vector in X. So as t goes to zero, perturbed $x + tu$ approches x from the direction u and we have the directional derivative

$$\frac{\mathrm{d}}{\mathrm{d}t} f(x) = \lim_{t \to 0} \frac{\|f(x + tu) - f(x)\|}{t}.$$

We know that this depend on u, but we can take is max over all u.

Writing δx for a generic perturbation tu that we add to x, we let

$$\delta f(x) = f(x + \delta x) - f(x).$$

The *absolute condition number* of the problem at x is

$$\hat{\kappa}(x) = \lim_{\delta \to 0} \sup_{\|\delta x\| \leq \delta} \frac{\|\delta f(x)\|}{\|\delta x\|}.$$

This is the max of the linear directional derivitives, and is shown in a multivariate calculus class to be the norm $\|J\|$ of the *Jacobian* matrix J whose entries are the partials derivatives $\partial f_i/\partial x_j$ of the componets of f.

More often, we are interested in the (relative) condition number of $f(x)$ with normalised error

$$\kappa(x) = \lim_{\delta\to 0} \sup_{\|\delta x\|\le\delta} \frac{\|\delta f(x)\|/\|f(x)\|}{\|\delta x\|/\|x\|} = \frac{\|J\|}{\|f(x)\|/\|x\|}.$$

We generally writes just $\sup_{\delta x}$ for $\lim_{\delta\to 0}\sup_{\|\delta x\|\le\delta}$, understanding by use of 'δ' that these are taken infinitesimally.

A problem is *well-conditioned* if κ is small, say $\kappa = 1$ or 100, and *ill-conditioned* if κ is big, say 10^5.

Example 3.2. Let $f(x) = \sqrt{x}$. The Jacobian is the 1×1 matrix $[\frac{\mathrm{d}}{\mathrm{d}x}\sqrt{x}] = [1/2\sqrt{x}]$. So

$$\hat{\kappa} = \frac{\|J\|}{\|f(x)\|/\|x\|} = \frac{1/2\sqrt{x}}{\sqrt{x}/x} = 1/2$$

We say that a problem is well-conditioned of all instances of the problem are well-contitioned, and say it is ill-conditioned if some instances are ill-conditioned. The problem of finding the eigenvalues of a matrix known to be ill-conditioned. (See the Trefethan Bau book for an example.)

We showed weeks ago that, for fixed A, the condition number of the problem of solving $Ax = b$ (so the problem $f(b) = x$) is the condition number $\|A\|\|A^{-1}\|$ of A. So is the problem $f(x) = b$ of finding b as a function of x.

Example 3.3. For fixed A the condition of finding Ax for an instance x we have that $A(x+\delta x) = Ax + A\delta x$ so $\delta b = A\delta x$.

$$\kappa = \sup_{\delta x} \frac{\|A\delta x\|/\|Ax\|}{\|\delta x\|/\|x\|} = \sup_{\delta x} \frac{\|A\delta x\|}{\|\delta x\|}\frac{\|x\|}{\|Ax\|} = \|A\|\frac{\|x\|}{\|Ax\|}.$$

Now if A is inverible this becomes

$$\kappa \le \|A\|\|A^{-1}\|$$

and the problem of Matrix-vector mutliplication, when the instance x is not specified, we get equality here by maximising over unit vectors x.

Practice

Find the condition number in A of the problem of solving $Ax = b$, for fixed b: $f(A) = x$.

3.2 Floating Point Arithmetic

Numbers in computers for real computations are usually represented as floating point numbers in some base β (almost always $\beta = 2$) upto some level of precision t (usually $t = 24$ or 53). This

means a number $r \in \mathbb{R}$ is represented as

$$r = \pm(m/\beta^t) * \beta^e$$

The 24 and 53 are the precision left from 32 or 64 bits after they take 1 bit for sign, and 8 or 11 for the exponent.

where m is an integer between 1 and β^t, and e is an integer. The set F of (precision t) *floating point numbers* consists of 0 and all numbers of the form $\pm(m/\beta^t) * \beta^e$.

Whatever the exponent e, our *fraction* or *mantissa* $\pm(m/\beta^t)$ is one of β^t discrete values

$$1 + \frac{1}{\beta^t}\{0, 1, 2, \ldots, \beta^t - 1\}$$

between 1 and 2. For any positive real r we get a unique *floating point representation* $\text{fl}(r)$ in F. We find the fraction by scaling it to r/β^e between 1 and 2, and then rounding to the closest element in F. A real number r will differ from $\text{fl}(r)$ a relative error of up to *machine epsilon*, $\varepsilon_{\text{mach}} := \beta^{-t}$:

$$\|\text{fl}(r)/r\| \leq 1 \pm \varepsilon_{\text{mach}}.$$

In this chapter we look at how much algorithms will blow up error. If we are adding n numbers that have $\varepsilon_{\text{mach}}$ error each, we cannot avoid solutions that will have up to $n\varepsilon_{\text{mach}}$ error.

We start with an assumption on the error or our basic operations. For any operation $* \in \{+, -, \times, \div\}$ let $\circledast$ be the operation on fixed point numbers F.

We will assume that the computer can compute these functions as accurately as possible over F. For any $x, y \in F$,

$$x \circledast y = \text{fl}(x * y). \tag{1}$$

As we can write $x, y \in F$ as $\text{fl}(x)$ and $\text{fl}(y)$, this identity, the *fundamental axiom of floating point arithmetic* can be viewed as a type of commutation of operations $\text{fl}(x) \circledast \text{fl}(y) = \text{fl}(x * y)$.

With the definition of $\varepsilon_{\text{mach}}$, it gives the useful corollary that for any basic operation $*$ and all $x, y \in X$ there is ε with $|\varepsilon| < \varepsilon_{\text{mach}}$ such that

$$x \circledast y = (x * y)(1 + \varepsilon). \tag{2}$$

Problem

Represent the numbers 1/2 and 3 as floating point numbers with base $\beta = 2$.

3.3 Stabililty

Where as conditioning was about problems, stabilitly is about the algorithms that solve them.

Adding $\varepsilon_{\text{mach}}$ relative error to A causes error in the solution to $Ax = b$. The ideal would be that we can find $\text{fl}(x)$ where x is the exact solution. How closely we acheive this depends on how we solve the problem.

Where $f : X \to Y$ is a problem, let $\tilde{f} : X \to Y$ be an algorithm for the problem that uses only floating point numbers.

An algorithm $\tilde{f}$ for a problem f is *accurate* if for all $x \in X$

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\varepsilon_{\text{mach}}).$$

This big-O notation means that as $\varepsilon_{\text{mach}}$ goes to 0, the relative error in computed $\tilde{f}(x)$ goes to 0– it is less than $C\varepsilon_{\text{mach}}$ for some fixed C. This is a theoretical measure of the accuracy of the algorithm. In practice, our hardware (or the architecture) fixes $\varepsilon_{\text{mach}}$. If C is big, approaching $1/\varepsilon_{\text{mach}}$ then the accurate algorithm is useless. But this is not the algorithm's fault. It is waiting there for better computers with even smaller $\varepsilon_{\text{mach}}$, and then it will be accurate in practice.

But generally, in this section though, the constants C of a $O(\varepsilon_{\text{mach}})$ is modest, less than 1000, for values of $\varepsilon_{\text{mach}}$ that we are interested in.

Accuracy, of course can be impossible for an ill-conditioned problem, and this will be no fault of the algorithm. Our first try at weakening this condition might be to ask for a function $\tilde{f}$ such that $\tilde{f}(\text{fl}(x))$ will always be close to $f(x)$. But this will also like fail at points x were f is ill-conditioned. To avoid this, we allow perturbing x to some nearby well-behaved $\tilde{x}$ and make the following definition.

We can assume the two constants for $O(\varepsilon_{\text{mach}})$ in the def of stability are the same– the second one is 'for some', which is forgiving.

An algorithm $\tilde{f}$ for a problem f is *stable* if for all $x \in X$

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = O(\varepsilon_{\text{mach}}) \qquad \text{for some } \tilde{x} \text{ with} \qquad \frac{\|\tilde{x} - x\|}{\|x\|} = O(\varepsilon_{\text{mach}}).$$

This

It is *backward stable* if

$$\tilde{f}(x) = f(\tilde{x}) \qquad \text{for some } \tilde{x} \text{ with} \qquad \frac{\|\tilde{x} - x\|}{\|x\|} = O(\varepsilon_{\text{mach}}).$$

The text gives a nice way to think of this– a problem is stable if it gives nearly the right answer to nearly the right question, it is backwards stable if it gives the right answer to nearly the right question.

All of these definitions talk of a functions with a vector x as input, but many of the functions we care about will have several vectors or matrices as input. This is okay, as these can all be viewed as a vector: the input x of the problem of compting the inner product of u and v can be viewed as the vector of vector $x = u|v$ we get by concatenating u and v. No matter which norm we are using, we get that $\|u|v\| \leq \|u\| + \|v\|$, so if we can say that $\|u\|, \|v\| \in O(\varepsilon_{\text{mach}})$ we get that $\|x\| \in O(\varepsilon_{\text{mach}})$.

The fundamental theorem of fixed point arithmetic asserts that the basic operations of addition, subtraction, multiplication, and division all have backwards stable algorithms.

Example 3.4. Where $f(a, b) = a - b$ is subtraction, we show that $\tilde{f}(a, b) = \text{fl}(a) \ominus \text{fl}(b)$ is backwards stable. Indeed there are $\varepsilon_a, \varepsilon_b < \varepsilon_{\text{mach}}$ such that

$$\text{fl}(a) = a(1 + \varepsilon_a) \qquad \text{and} \qquad \text{fl}(b) = b(1 + \varepsilon_b).$$

So taking $\tilde{a} = \text{fl}(a)$ and $\tilde{b} = \text{fl}(b)$ we thus have that

$$\frac{\|\tilde{a} - a\|}{\|a\|} = O(\varepsilon_{\text{mach}}) \qquad \text{and} \qquad \frac{\|\tilde{b} - b\|}{\|b\|} = O(\varepsilon_{\text{mach}}).$$

Moreover, by (1) we have that

$$\tilde{f}(a,b) = \mathrm{fl}(a) \ominus \mathrm{fl}(b) = \mathrm{fl}(\mathrm{fl}(a) - \mathrm{fl}(b)) = f(\tilde{a}, \tilde{b}),$$

showing that $\tilde{b}$ is backwards stable.

Problem

Show that the sum $\bigoplus a_i$ of several numbers is backwards stable. Show that floating point division is backwards stable. (This is perhaps surprizing. It is even stable then the denominator is 0.)

Example 3.5. The floating point inner product

$$\tilde{f}(a,b) = \bigoplus(\mathrm{fl}(a_i) \otimes \mathrm{fl}(b_i)) = (\mathrm{fl}(a_1) \otimes \mathrm{fl}(b_1)) \oplus \cdots \oplus (\mathrm{fl}(a_n) \otimes \mathrm{fl}(b_n))$$

of two vectors is backwards stable. Again taking $\tilde{x} = \mathrm{fl}(x)$ for each of the variables a_i and b_i, let x be the vector of all input variables, and $\tilde{x}$ be corresponding vector of their floating point reducitons. One gets

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\varepsilon_{\mathrm{mach}}).$$

To show that $\tilde{f}$ is backwards stable, we show that

$$\bigoplus(\tilde{a}_i \otimes \tilde{b}_i) = \sum \tilde{a}_i \times \tilde{b}_i.$$

By the (1) we have that $\tilde{a}_i \otimes \tilde{b}_i = \mathrm{fl}(\tilde{a}_i \times \tilde{b}_i)$ for all i, and using it again we get that

$$\bigoplus(\tilde{a}_i \otimes \tilde{b}_i) = \bigoplus \mathrm{fl}(\tilde{a}_i \times \tilde{b}_i) = \mathrm{fl}(\sum \tilde{a}_i \times \tilde{b}_i) = \sum \tilde{a}_i \times \tilde{b}_i.$$

It is tempting to guess that the composition of backwards stable functions is stable, but this is not true. It is for stable functions though.

Example 3.6. Consider the obvious floating point algorithm $\tilde{f}$ for the outer-product problem $f(uv) = uv^T$. It would return a matrix $A = [a_{ij}]$ with $a_{ij} = u_i \otimes v_j$. This algorithm is stable, as A is less than $n^2\varepsilon_{\mathrm{mach}}$ from uv^T. But A generally has high rank. We cannot write it as the rank one matrix $\tilde{u}\tilde{v}^T$ for any $\tilde{u}$ and $\tilde{v}$. So the algorithm is not backwards stable.

We note that the norm that we are using is not important to the definition of accuracy or stability. Indeed, we know that for any two norms $\|\cdot\|$ and $\|\cdot\|_*$ of $\mathbb{R}^n$ there are constants c_1 and c_2 such that for all $x \in \mathbb{R}^n$,

$$\|x\|c_1 \leq \|x\|_* \leq \|x\|c_2.$$

Problem

Show that an algorithm that is accurate, stable, or backwards stable with respect to the 2-norm is accurate, stable or backwards stable with respect to you other favourite norm.

When an algorithm $\tilde{f}$ for a problem f is backwards stable, the accuracy $\tilde{f}$ of the algorithm is controlled by the relative condition number κ of f.

Indeed we have the following.

Theorem 3.7. *Let $\tilde{f}$ be a backwards stable algorithm for a problem $f : X \to Y$ with condition number κ. For any x in X*

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|x\|} = O(\kappa \varepsilon_{\text{mach}}).$$

Problem

Prove this theorem.

Without the precise definitions, we saw earlier that Gram Schmid is not very stable, and certainly not backwards stable. Permuting columns improves a bit, but it is still not backward stable.

In the rest of this chapter, it is shown that the Householder algorithm for computing the QR-decomposition of a matrix is backwards stable. What this means is interesting.

The QR-decomposition $A = QR$ we get from Gauss Schmidt or Householder, if there is no error, will gives essentially the same Q and R. But in practice, either algorithm will give some error, and this error will be different. When we find a QR-decomposition $A \approx Q^*R^*$ via the Householder algorithm, backwards stability means that there is some A^* close to A such that $A^* = Q^*R^*$. In the text they show by example that Q^* and R^* can be relativley far from Q and R, where $A = QR$. So if our goal it really to find Q^* and R^*, this is not so good. But our goal is probably to use them to compute the solution to $Ax = b$, and for this they can be quite useful– their acuracy in computing this depends on the relative conditon number of A.

We skipped it before, but perhaps now that we see why, we can look at the Householder algorithm for computing $A = QR$.

3.3.1 The Householder Algorithm

We now get a QR-decompostion of $QR = A = [\,a_1|\cdots|a_n\,]$ by multiplying A on the left by several orthogonal matrices $Q_1, \ldots, Q_n$ such that

$$Q_n Q_{n-1} \ldots Q_1 A =: R$$

is upper triangular.

The first step in triangulating A is to find orthogonal matrix $Q_1 = H_1$ such that

$$A_1 = Q_1 A = \left[\begin{array}{c|c} \sigma_1 & ? \\ \hline 0 & \\ 0 & ? \\ 0 & \end{array}\right]$$

where we have eliminated under the first entry. We do so in such a way that $\sigma_1 = \pm\|a_1\|$. If we can find an orthogonal matrix H_a for any vector a such that $H_a a$ is this column, then we are happy.

Taking $Q_i = I_i \oplus H_{b_i}$ where b_i is vector of the bottom $n-i$ entries of a_i we get that

$$Q_2 A_1 = Q_2 Q_1 A = \left[\begin{array}{c|ccc} 1 & 0 & 0 & 0 \\ \hline 0 & & & \\ 0 & & H_2 & \\ 0 & & & \end{array}\right] \left[\begin{array}{c|c} \sigma_1 & ? \\ \hline 0 & \\ 0 & ? \\ 0 & \end{array}\right] = \left[\begin{array}{cc|c} \sigma_1 & ? & ? \\ 0 & \sigma_2 & \\ \hline & & \\ 0 & 0 & ? \\ & & \end{array}\right].$$

Continuing in this way, we will get upper diagonal R, and as the product of orthogonal matrices is orthogonal $Q = Q_n Q_{n-1} \dots Q_1$ will be the Q we want.

The matrix H_a we will use (or $-H_a$) is called the *Householder reflection* of a. An orthogonal matrix such that $-H_a a = \|a\| e_1$. This reflects a onto the x axis through the hyperspace that bisects the vector $v = a - \|a\| e_1$. (FIGURE HERE). Indeed, for vector a let $\sigma = \|a\|$ and $v = a - \sigma e_1$. Let

$$H = H_a := I - 2P_v = I - 2\frac{vv^T}{v^T v}.$$

Problem

Prove the following.

Lemma 3.8. *Let H_a be the Householder reflection for a vector a. The following are true.*

i. $H_a a = -\|a\| e_1$

ii. H_a *is invertible*

iii. H_a *is symmetric*

iv. H_a *is orthogonal*

One might observe that using $-H_a$ in place of H_a will accomplish the same goal: $-H_a a$ is also a column with a non-zero element on the top of a bunch of zeros. And this is true, but $H_a a$ is further from a than $-H_a a$ is, and this adds stability to the algorithm.

Problem

We have only outlined the algorithm. Give a nice concise pseudo-code version of the Householder algorithm to find a QR decomposition of A.

Problems from the text

Lectures 14_{TF} and 15_{TF}: 14.1(f,g), 14.2, 15.1(a,c)

IV Low Rank and Compressed Sensing

While the matrices use in Data applications are big, the are usually 'secretly small' being low rank, or sparse, or having only a few significant singular values This chapter is about this and the tools that let us take advantage of these ideas.

The first two sections are about how low rank changes in a matrix A yield only small changes in some of the interesting problems related to A. We see how a low rank change in A yields a low rank change in A^1 and so yields only small changes in solving $Ax = b$; and then how low rank changes yield only controlled changes in eigenvalues. Both of these ideas come in handing in application when we want to make changes– we don't have to completely recompute everything.

Section three is about matrics in which the singular values decay, so only very few of them are significant, so the matrix has low effective rank.

We have only talked briefly about sparsity, but it is, like low rank, a way that big matrices can be secretly small. Low rank means that there are only few important directions. Sparsity means that there are only a few important coordinates. The last two sections start make ideas of sparsity clear. Compressed sensing is about finding sparse solutions x to $Ax = b$. Will be surprised, when we look at matrix completion, at how well a matrix can be recovered from just a few entries.

IV.1 Changes in A^{-1} from changes in A

The matrix inversion formula, or the Sherman-Morrison-Woodbury formula shows us how to recompute the inverse of a matrix when we change just a couple of columns, or more generally, add a low rank matrix to it. This can be very useful in applications where we are solving equations and then making small adjustments– computing the inverse is the hard part of solving $Ax = b$.

Theorem IV.1 (Matrix inversion formula)**.** *For an invertible A and an $n \times n$ rank r matrix UV^T if $M = A - UV^T$ is invertible, then the inverse is*

$$M^{-1} = A^{-1} + A^{-1}U(I - V^T A^{-1}U)^{-1}V^T A^{-1}.$$

One can only compute the formula if the $r \times r$ matrix $(I - V^T A^{-1}U)$ is invertible, and if we alreay have A^{-1} then we can compute M^{-1} by from the inverse of this by matrix multiplications.

Practice

Show that computing M^{-1} using the matrix inversion formula take $O(rn^2)$ operations, compared to the $O(n^3)$ needed to compute M^{-1} directly.

One can show that the matrix inversion formula is true by multiplying out

$$(A - UV^T)(A^{-1} + A^{-1}U(I - V^T A^{-1}U)^{-1}V^T A^{-1}).$$

The text gives another nice proof.

Practice

Look at the proof in the text, at least for the case that $A = I$ and UV^T has rank 1. This is given on p. 160-161. Details are left to Problem 2. Fill these in.

Now assume that we have found the least squares solution $\hat{x}$ for some $Ax = b$, so we have that $\hat{x}$ satisfies the normal equation

$$A^T Ax = A^T b$$

and we have found $\hat{x}$ by computing $(A^T A)^{-1}$ and $A^T b$.

We add another equation to the system of equations as an extra row to the equation $Ax = b$. This means we have a new matrix $A' = \begin{bmatrix} A \\ r \end{bmatrix}$ that we get from A by appending a new row r; at the same time b gets one one more $m + 1^{st}$ coordinate. The normal equation becomes

$$\begin{bmatrix} A^T & r^T \end{bmatrix}\begin{bmatrix} A \\ r \end{bmatrix}x = \begin{bmatrix} A^T & r^T \end{bmatrix}\begin{bmatrix} b \\ b_{m+1} \end{bmatrix}$$

which can be written

$$(A^T A + r^T r)x = A^T b + r^T b_{m+1}.$$

We have added the rank one matrix $r^T r$ to $A^T A$, so we can find our new $\hat{x}$ by updating $(A^T A)^{-1}$ and the target vector $A^T b + r^T b_{m+1}$. The latter part is the non-trivial bit, but we can do it with the matrix inversion formula on $A^T A$ in place of A, with $U = -r^T$ and $V^T = r$.

$$\begin{aligned}(A^T A + r^T r)^{-1} &= (A^T A)^{-1} - A^T A^{-1} r^T [I + r(A^T A)^{-1} r^T]^{-1} r(A^T A)^{-1} \\ &= (A^T A)^{-1} - \frac{A^T A^{-1} r^T r (A^T A)^{-1}}{(1 + r(A^T A)^{-1} r^T)}\end{aligned}$$

The second equality uses ii) from the following exercise with r as B and $(A^T A)^{-1} r^T$ as A, to get

$$[I + r(A^T A)^{-1} r^T]^{-1} r = r[1 + r(A^T A)^{-1} r^T]^{-1}.$$

The right side of this is scaler because, recall r is a row, so $r(A^T A)^{-1} r^T$ is a column.

Strang lists the following identities innocuously on the page before this proof, and uses them without mention. He's tricky that way.

Practice

Show that the following are true when the necessary inverses exist.

i. $B(I_m + AB) = (I_n + BA)B$

ii. $B(I_m + AB)^{-1} = (I_n + BA)^{-1} B$

iii. $A^T (AA^T + \lambda I_n)^{-1} = (A^T A + \lambda I_m)^{-1} A^T$

iv. $U(I_k - V^T U) = (I_n - UV^T)U$

Attacking least squares this way, updating when we add new constraints, is called *recursive least squares*. We can add more than one row at a time, and the same idea works.

Problems from the text

III.1: 1-5

V Sample problems for the final

Here are typical problems, very much like those you will encounter on the Final.

i. Explain how you use a QR-decomposition of A to solve $Ax = b$.

ii. Give the pseudo inverse of $A = \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 2 & 3 \end{bmatrix}$.

iii. Give the pseudo inverse of $A = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 2 & 3 \end{bmatrix}$.

iv. Write out the algorithm for Gram Schmid with pivoting. (You may write it as finding an orthonormal basis Q equivalent to a basis B, or as an algorithm for finding a QR-decomposition of a matrix A.)

v. Explain what the benefit of (column) pivoting is when doing Gram Schmidt.

vi. Explain, in detail, how you would approximately compute the the main eigenvector of a matrix A.

vii. True or false.

(a) If $\tilde{f}$ is stable, then $\tilde{f}^{-1}$ is backwards stable.

(b) $\text{fl}(\pi) - \pi = O(\varepsilon_{\text{mach}})$.

(c) The algorithm $\tilde{f}(x) = x \oplus x$ is backwards stable.

viii. Find the condition number of the problem $f(x, y) = xy$.

ix. Represent the numbers $1/2$ and 3 as floating point numbers with base $\beta = 2$.

x. Give the precision $t = 5$ decimal ($\beta = 10$) floating point representation of the number $1/3$.

xi. In the floating point represenation $\pm(m/\beta^t) * \beta^e$ of a real number r what is the integer e? What is it when $t = 5$ and $r = 43.292285$?

xii. Give (with explanation) an example of an algorithm for a problem (which correctly solves the problem when $\varepsilon_{\text{mach}} = 0$) that is not backwards stable.

xiii. Show that the floating point sum $\tilde{f}(a, b, c) = \text{fl}(a) + \text{fl}(b) + \text{fl}(c)$ is a backwards stable algorithm for $f(a, b, c) = a + b + c$.